TP1 - Découverte de graphes causaux sur le jeu de données Dépression

Cours: SDD 1004
Membres de l'équipe:

- **FONE SIMO Inzaghi Merlin FONI77310300**
- **TCHUINKWA TANDU Jeriel TCHJ85330100**

Table des matières

1. Prétraitement des données
2. DAG-GNN : Expérimentation de paramètres
3. Comparaison avec deux autres algorithmes
4. Algorithme non mentionné dans le mémoire
5. Conclusion

In [13]:

```
pip install gcastle --break-system-packages
```

```
Requirement already satisfied: gcastle in /usr/local/lib/python3.12/dist-packages (1.0.4)
Requirement already satisfied: matplotlib>=2.1.2 in /usr/local/lib/python3.12/dist-packag
es (from gcastle) (3.10.0)
Requirement already satisfied: networkx>=2.5 in /usr/local/lib/python3.12/dist-packages (
from gcastle) (3.6.1)
Requirement already satisfied: numpy>=1.19.1 in /usr/local/lib/python3.12/dist-packages (
from gcastle) (2.0.2)
Requirement already satisfied: pandas>=0.22.0 in /usr/local/lib/python3.12/dist-packages
(from gcastle) (2.2.2)
Requirement already satisfied: scikit-learn>=0.21.1 in /usr/local/lib/python3.12/dist-pac
kages (from gcastle) (1.6.1)
Requirement already satisfied: scipy>=1.7.3 in /usr/local/lib/python3.12/dist-packages (f
rom gcastle) (1.16.3)
Requirement already satisfied: tqdm>=4.48.2 in /usr/local/lib/python3.12/dist-packages (f
rom gcastle) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-package
s (from matplotlib>=2.1.2->gcastle) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (f
rom matplotlib>=2.1.2->gcastle) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packag
es (from matplotlib>=2.1.2->gcastle) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packag
es (from matplotlib>=2.1.2->gcastle) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages
(from matplotlib>=2.1.2->gcastle) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from
matplotlib>=2.1.2->gcastle) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-package
s (from matplotlib>=2.1.2->gcastle) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-pac
kages (from matplotlib>=2.1.2->gcastle) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (f
rom pandas>=0.22.0->gcastle) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
(from pandas>=0.22.0->gcastle) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (
from scikit-learn>=0.21.1->gcastle) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-pac
kages (from scikit-learn>=0.21.1->gcastle) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from
python-dateutil>=2.7->matplotlib>=2.1.2->gcastle) (1.17.0)
```

Imports et Configuration

In [14]:

```
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
import torch
from castle.algorithms import DAG_GNN, PC
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

# Configuration matplotlib
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print("✓ Imports réussis")
print(f"✓ PyTorch version: {torch.__version__}")
print(f"✓ Device disponible: {'GPU (MPS)' if torch.backends.mps.is_available() else 'CPU'
}")
```

```
✓ Imports réussis
✓ PyTorch version: 2.10.0+cpu
✓ Device disponible: CPU
```

1. Prétraitement des données

Nous appliquons le même prétraitement que dans le code fourni :

- **Échantillonnage de 10% des données**
- **Regroupement des statuts maritaux**
- **Transformation de "Number of Children" en variable binaire**
- **Encodage ordinal des variables comportementales**
- **Normalisation StandardScaler**

In [15]:

```
def load_and_preprocess_data(file_path='depression.csv', sample_frac=0.1):
    """
    Charge et prétraite les données de dépression
    """
    print("Chargement des données...")

    # Charger (exclure 'Name')
    df = pd.read_csv(file_path, usecols=lambda col: col != 'Name')
    df = df.sample(frac=sample_frac, random_state=42)

    print(f"    Shape: {df.shape}")

    # Regrouper statuts maritaux
    df['Marital Status'] = df['Marital Status'].replace({
        'Divorced': 'Single/Divorced/Widowed',
        'Widowed': 'Single/Divorced/Widowed',
        'Single': 'Single/Divorced/Widowed'
    })

    # Transformer 'Number of Children' en binaire 'Has Children'
    df['Number of Children'] = df['Number of Children'].apply(lambda x: 1 if x > 0 else
0)
    df = df.rename(columns={'Number of Children': 'Has Children'})

    # Mappings ordinaux
```

```
    mappings = {
        'Dietary Habits': {'Unhealthy': 1, 'Moderate': 2, 'Healthy': 3},
        'Sleep Patterns': {'Poor': 1, 'Fair': 2, 'Good': 3},
        'Physical Activity Level': {'Sedentary': 1, 'Moderate': 2, 'Active': 3},
        'Education Level': {'High School': 1, 'Associate Degree': 2,
                            "Bachelor's Degree": 3, "Master's Degree": 4, 'PhD': 5},
        'Alcohol Consumption': {'Low': 1, 'Moderate': 2, 'High': 3},
        'Smoking Status': {'Non-smoker': 1, 'Former': 2, 'Current': 3},
        'Chronic Medical Conditions': {'No': 1, 'Yes': 2},
    }

    for column, mapping in mappings.items():
        if column in df.columns:
            df[column] = df[column].map(mapping)

    # Encoder les catégorielles restantes
    categorical_columns = df.select_dtypes(include=['object']).columns
    df[categorical_columns] = df[categorical_columns].astype('category').apply(lambda x:
x.cat.codes)

    scaler = StandardScaler()
    df[df.columns] = scaler.fit_transform(df[df.columns])

    print("   ✓ Prétraitement terminé")

    return df

# Charger les données
df = load_and_preprocess_data()
column_names = df.columns.tolist()

print(f"\nVariables ({len(column_names)}):")
for i, col in enumerate(column_names, 1):
    print(f"  {i:2d}. {col}")
```

```
Chargement des données...
    Shape: (41377, 15)
   ✓ Prétraitement terminé

Variables (15):
    1. Age
    2. Marital Status
    3. Education Level
    4. Has Children
    5. Smoking Status
    6. Physical Activity Level
    7. Employment Status
    8. Income
    9. Alcohol Consumption
   10. Dietary Habits
   11. Sleep Patterns
   12. History of Mental Illness
   13. History of Substance Abuse
   14. Family History of Depression
   15. Chronic Medical Conditions
```

Fonction de visualisation des DAG

In [16]:

```
def visualize_dag(adj_matrix, column_names, title="Graphe Causal", threshold=0.0):
    """
    Visualise le DAG avec des FLÈCHES TRÈS VISIBLES (sans pygraphviz)
    """
    # Appliquer le seuil
    adj_thresholded = np.where(np.abs(adj_matrix) > threshold, adj_matrix, 0)

    graph = nx.from_numpy_array(adj_thresholded, create_using=nx.DiGraph)
    labels = {i: column_names[i] for i in range(len(column_names))}
```

```
    plt.figure(figsize=(18, 12))

    # Layout : Kamada-Kawai (bien espacé) ou Spring
    if len(graph.nodes()) > 0:
        pos = nx.kamada_kawai_layout(graph)
    else:
        pos = nx.spring_layout(graph, k=3, iterations=50, seed=42)

    # Dessiner les NŒUDS (plus grands)
    nx.draw_networkx_nodes(
        graph, pos,
        node_size=5000,           # Très grands
        node_color='lightblue',
        alpha=0.95,
        edgecolors='darkblue',    # Bordure bleue foncée
        linewidths=3
    )

    # Dessiner les LABELS
    nx.draw_networkx_labels(
        graph, pos, labels,
        font_size=10,
        font_weight='bold',
        font_color='black'
    )


    nx.draw_networkx_edges(
        graph, pos,
        arrows=True,
        arrowsize=40,              # ÉNORME
        arrowstyle='-|>',          # Style flèche triangulaire
        edge_color='crimson',      # ROUGE FONCÉ (bien visible)
        width=4,                   # TRÈS ÉPAIS
        alpha=0.9,
        connectionstyle='arc3,rad=0.2',  # Courbure pour éviter chevauchements
        node_size=5000,            # Doit correspondre à node_size ci-dessus
        min_source_margin=40,      # Espace entre nœud et début de flèche
        min_target_margin=40       # Espace entre fin de flèche et nœud
    )

    plt.title(title, fontsize=20, fontweight='bold', pad=30, color='darkblue')
    plt.axis('off')
    plt.margins(0.15)  # Marges pour que tout soit visible
    plt.tight_layout()
    plt.show()

    # Statistiques
    num_edges = np.sum(np.abs(adj_thresholded) > 0)
    num_nodes = len(column_names)
    density = num_edges / (num_nodes * (num_nodes - 1)) if num_nodes > 1 else 0

    print(f"\n Statistiques du graphe:")
    print(f"    • Nœuds: {num_nodes}")
    print(f"    • Arêtes: {num_edges}")
    print(f"    • Densité: {density:.3f}")

    # Afficher quelques arêtes
    if num_edges > 0 and num_edges <= 20:
        print(f"\n Arêtes détectées:")
        edge_list = list(graph.edges())
        for src, tgt in edge_list[:10]:
            print(f"   {column_names[src]} → {column_names[tgt]}")
        if len(edge_list) > 10:
            print(f"   ... et {len(edge_list) - 10} autres")

    return num_edges, density
```

2. DAG-GNN : Expérimentation de paramètres

Nous allons tester 4 modifications de paramètres et comparer les résultats. la configuration de base que nous avons choisi pour le DAGNN est baseline_config = { 'epochs': 1, 'lr': 0.005, 'optimizer': 'adam', 'graph_threshold': 0.1 } exactement les configurations que a utilisé Faustin Kagabo dans son rapport

2.1 Configuration de référence (Baseline)

In [17]:

```
print("="*70)
print("CONFIGURATION DE RÉFÉRENCE")
print("="*70)

baseline_config = {
    'epochs': 1,
    'lr': 0.005,
    'optimizer': 'adam',
    'graph_threshold': 0.1
}

print(f"Paramètres: {baseline_config}")

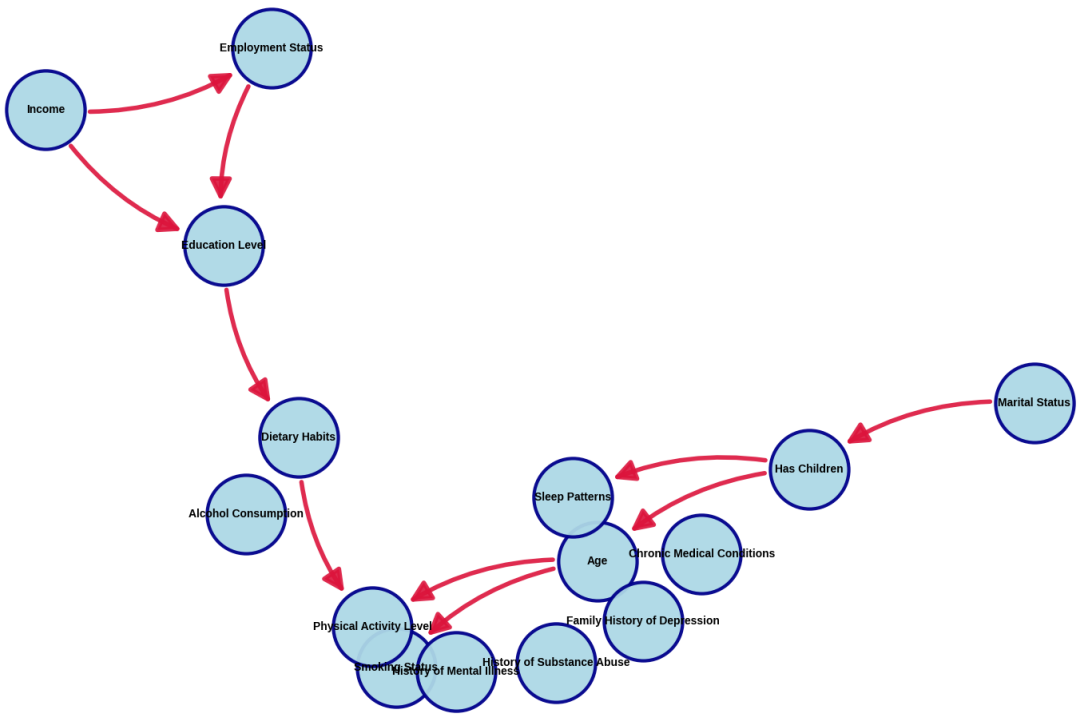
data_numpy = df.to_numpy(dtype=np.float32)

gnn_baseline = DAG_GNN(**baseline_config)
gnn_baseline.learn(data_numpy)

adj_baseline = gnn_baseline.causal_matrix
edges_baseline, density_baseline = visualize_dag(adj_baseline, column_names,
                                                 "DAG-GNN - Configuration de Référence"
)
```

```
======================================================================
CONFIGURATION DE RÉFÉRENCE
======================================================================
Paramètres: {'epochs': 1, 'lr': 0.005, 'optimizer': 'adam', 'graph_threshold': 0.1}
```

DAG-GNN - Configuration de Référence



Statistiques du graphe:

- Nœuds: 15
- Arêtes: 10

```
  • Arêtes: 10
  • Densité: 0.048

 Arêtes détectées:
   Age → Smoking Status
   Age → Physical Activity Level
   Marital Status → Has Children
   Education Level → Dietary Habits
   Has Children → Age
   Has Children → Sleep Patterns
   Employment Status → Education Level
   Income → Education Level
   Income → Employment Status
   Dietary Habits → Physical Activity Level
```

2.2 Expérience 1 : Augmentation du nombre d'époques

Paramètre modifié : `epochs = 30` **(au lieu de 1)**

Impact attendu :

- **Meilleure convergence de l'algorithme**
- **Graphe potentiellement plus stable**
- **Risque de sur-apprentissage si trop élevé**

In [18]:

```
print("="*70)
print("EXPÉRIENCE 1 : EPOCHS = 30")
print("="*70)

config_epochs = baseline_config.copy()
config_epochs['epochs'] = 3

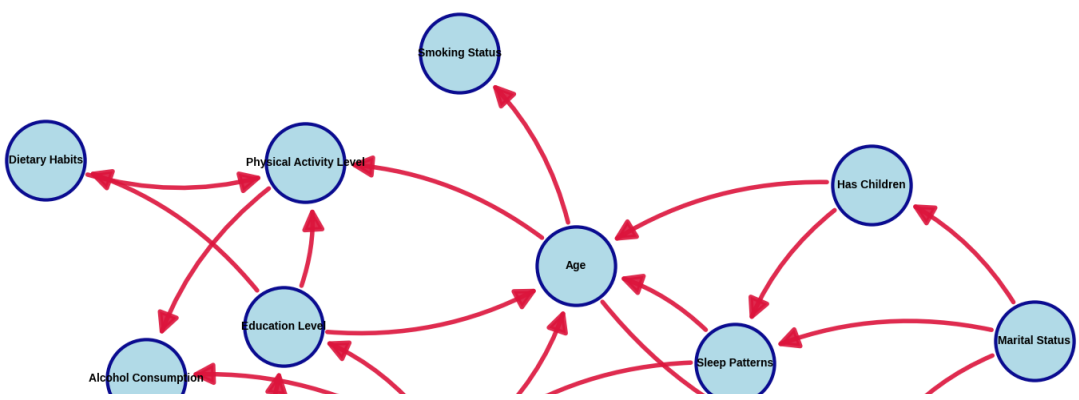
print(f"Paramètres: {config_epochs}")

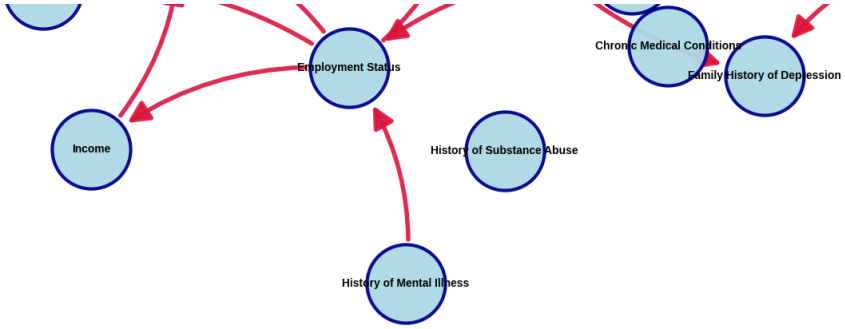
gnn_epochs = DAG_GNN(**config_epochs)
gnn_epochs.learn(data_numpy)

adj_epochs = gnn_epochs.causal_matrix
edges_epochs, density_epochs = visualize_dag(adj_epochs, column_names,
                                             "DAG-GNN - Epochs = 30")

print(f"\n Comparaison avec baseline:")
print(f"   Baseline: {edges_baseline} arêtes")
print(f"   Epochs=20: {edges_epochs} arêtes")
print(f"   Différence: {edges_epochs - edges_baseline:+d} arêtes")
```

```
======================================================================
EXPÉRIENCE 1 : EPOCHS = 30
======================================================================
Paramètres: {'epochs': 3, 'lr': 0.005, 'optimizer': 'adam', 'graph_threshold': 0.1}
```






```
Statistiques du graphe:
  • Nœuds: 15
  • Arêtes: 21
  • Densité: 0.100

Comparaison avec baseline:
  Baseline: 10 arêtes
  Epochs=20: 21 arêtes
  Différence: +11 arêtes
```

2.3 Expérience 2 : Réduction du learning rate

Paramètre modifié : `lr = 0.0001` **(au lieu de 0.001)**

Impact attendu :

- **Convergence plus lente mais plus stable**
- **Graphe potentiellement plus précis**
- **Peut nécessiter plus d'époques**

In [19]:

```
print("="*70)
print("□ EXPÉRIENCE 2 : LEARNING RATE = 0.0001")
print("="*70)

config_lr = baseline_config.copy()
config_lr['lr'] = 0.0001

print(f"Paramètres: {config_lr}")

gnn_lr = DAG_GNN(**config_lr)
gnn_lr.learn(data_numpy)

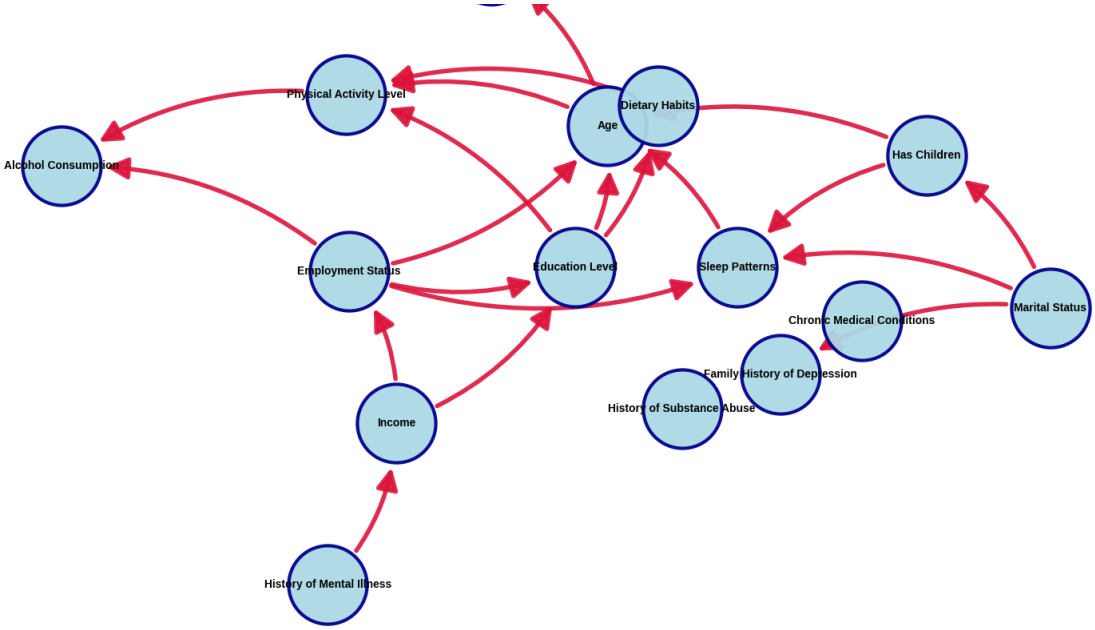
adj_lr = gnn_lr.causal_matrix
edges_lr, density_lr = visualize_dag(adj_lr, column_names,
                                     "DAG-GNN - Learning Rate = 0.0001")

print(f"\n□ Comparaison avec baseline:")
print(f"   Baseline: {edges_baseline} arêtes")
print(f"   LR=0.0001: {edges_lr} arêtes")
print(f"   Différence: {edges_lr - edges_baseline:+d} arêtes")
```

```
======================================================================
□ EXPÉRIENCE 2 : LEARNING RATE = 0.0001
======================================================================
Paramètres: {'epochs': 1, 'lr': 0.0001, 'optimizer': 'adam', 'graph_threshold': 0.1}
```

DAG-GNN - Learning Rate = 0.0001






```
Statistiques du graphe:
  • Nœuds: 15
  • Arêtes: 20
  • Densité: 0.095

Arêtes détectées:
  Age → Smoking Status
  Age → Physical Activity Level
  Marital Status → Has Children
  Marital Status → Sleep Patterns
  Marital Status → Family History of Depression
  Education Level → Age
  Education Level → Physical Activity Level
  Education Level → Dietary Habits
  Has Children → Age
  Has Children → Sleep Patterns
  ... et 10 autres

 Comparaison avec baseline:
  Baseline: 10 arêtes
  LR=0.0001: 20 arêtes
  Différence: +10 arêtes
```

2.4 Expérience 3 : Augmentation du seuil du graphe

Paramètre modifié : `graph_threshold = 0.6` **(au lieu de 0.1)**

Impact attendu :

- **Graphe plus parcimonieux (sparse)**
- **Moins d'arêtes (seules les plus fortes sont conservées)**
- **Structure causale plus claire**

In [20]:

```
print("="*70)
print("EXPÉRIENCE 3 : GRAPH THRESHOLD = 0.6")
print("="*70)

config_threshold = baseline_config.copy()
config_threshold['graph_threshold'] = 0.6

print(f"Paramètres: {config_threshold}")

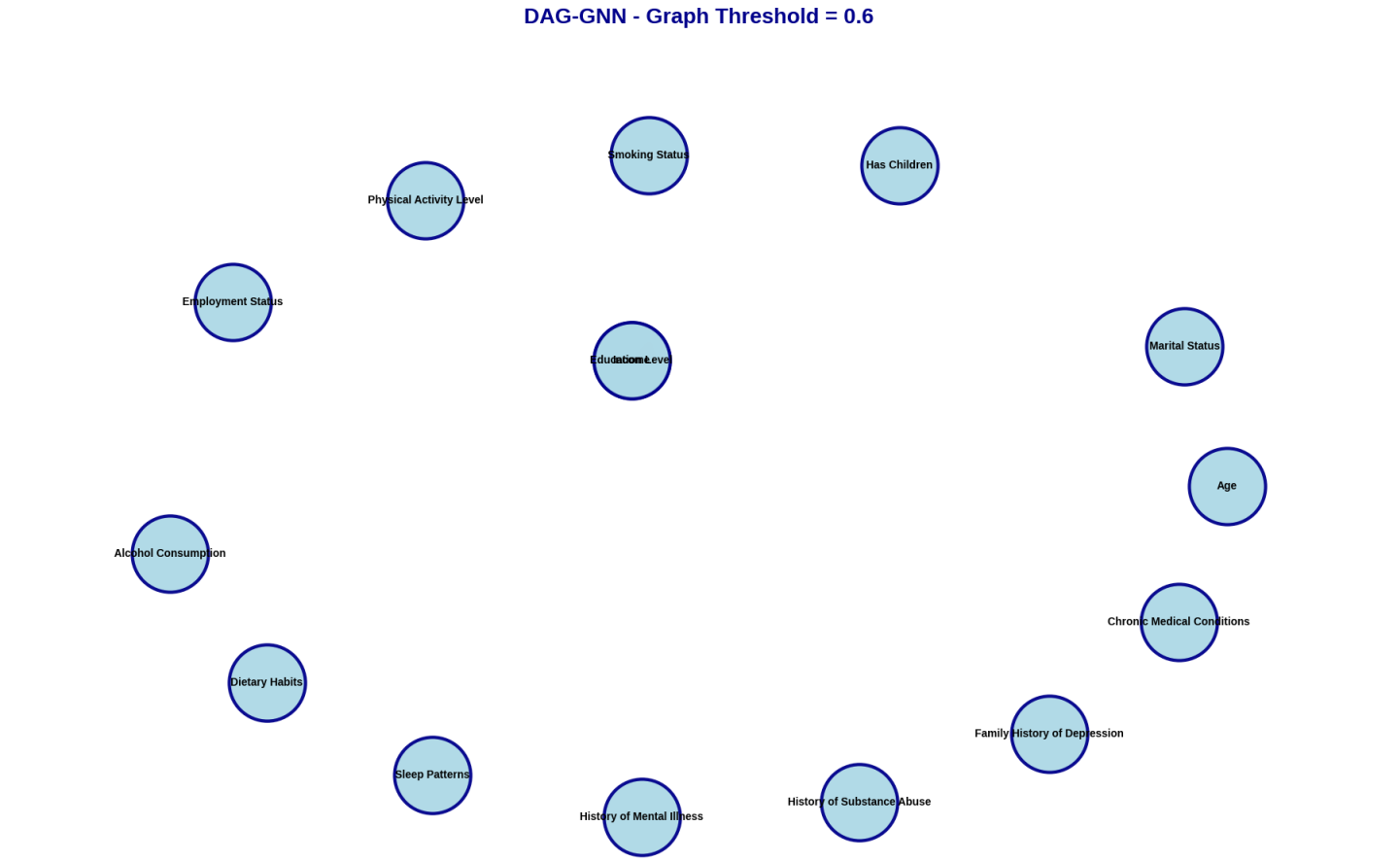
gnn_threshold = DAG_GNN(**config_threshold)
```

```
gnn_threshold.learn(data_numpy)

adj_threshold = gnn_threshold.causal_matrix
edges_threshold, density_threshold = visualize_dag(adj_threshold, column_names,
                                                   "DAG-GNN - Graph Threshold = 0.6")

print(f"\n Comparaison avec baseline:")
print(f"   Baseline: {edges_baseline} arêtes")
print(f"   Threshold=0.6: {edges_threshold} arêtes")
print(f"   Différence: {edges_threshold - edges_baseline:+d} arêtes")
```

```
======================================================================
EXPÉRIENCE 3 : GRAPH THRESHOLD = 0.6
======================================================================
Paramètres: {'epochs': 1, 'lr': 0.005, 'optimizer': 'adam', 'graph_threshold': 0.6}
```




```
 Statistiques du graphe:
   • Nœuds: 15
   • Arêtes: 1
   • Densité: 0.005

 Arêtes détectées:
   Income → Education Level

 Comparaison avec baseline:
   Baseline: 10 arêtes
   Threshold=0.6: 1 arêtes
   Différence: -9 arêtes
```

2.6 Synthèse des expériences DAG-GNN

In [21]:

```
# Tableau comparatif
results_df = pd.DataFrame({
    'Configuration': ['Baseline', 'Epochs=20', 'LR=0.001', 'Threshold=0.6'],
    'Nombre d\'arêtes': [edges_baseline, edges_epochs, edges_lr, edges_threshold],
    'Densité': [density_baseline, density_epochs, density_lr, density_threshold]
})
```

```
print("\n" + "="*70)
print("TABLEAU RÉCAPITULATIF DES EXPÉRIENCES DAG-GNN")
print("="*70)
print(results_df.to_string(index=False))

# Visualisation
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

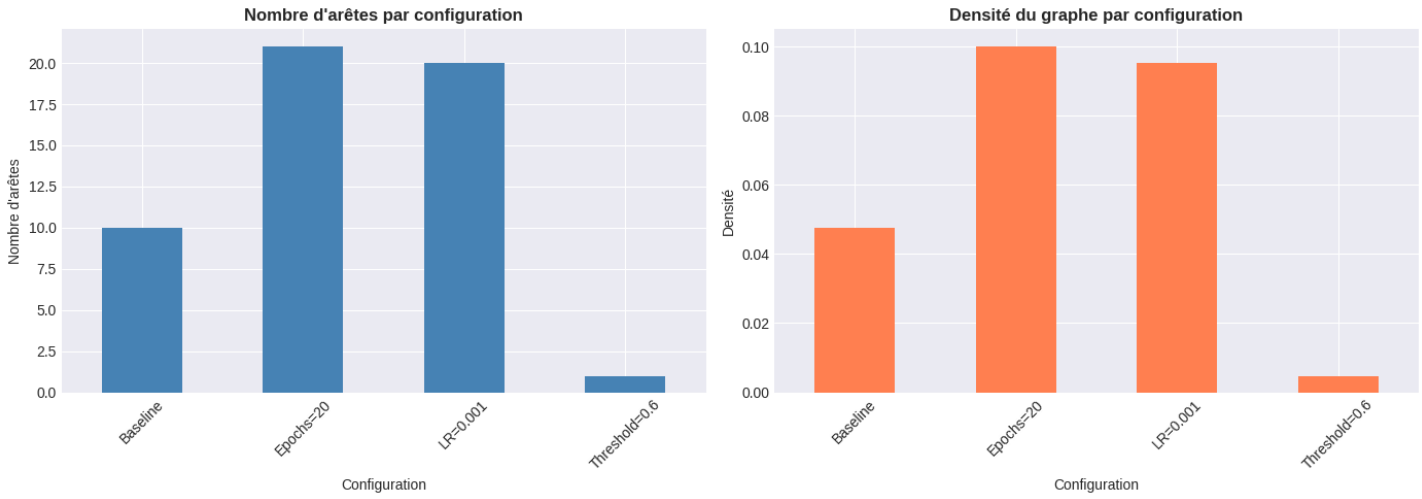
results_df.plot(x='Configuration', y='Nombre d\'arêtes', kind='bar', ax=ax1, color='steelblue', legend=False)
ax1.set_title('Nombre d\'arêtes par configuration', fontweight='bold')
ax1.set_ylabel('Nombre d\'arêtes')
ax1.tick_params(axis='x', rotation=45)

results_df.plot(x='Configuration', y='Densité', kind='bar', ax=ax2, color='coral', legend=False)
ax2.set_title('Densité du graphe par configuration', fontweight='bold')
ax2.set_ylabel('Densité')
ax2.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

print("\n Section DAG-GNN terminée")
```

```
======================================================================
TABLEAU RÉCAPITULATIF DES EXPÉRIENCES DAG-GNN
======================================================================
Configuration  Nombre d'arêtes  Densité
     Baseline               10 0.047619
    Epochs=20               21 0.100000
     LR=0.001               20 0.095238
Threshold=0.6                1 0.004762
```




```
 Section DAG-GNN terminée
```

3. Comparaison avec deux autres algorithmes

Nous allons maintenant comparer DAG-GNN avec :

1. **PC (Peter-Clark) : Algorithme basé sur les tests d'indépendance conditionnelle**
2. **NOTEARS : Formulation continue de la découverte causale**

3.1 : PC Algorithm (Peter-Clark)

Algorithme de découverte causale basé sur les tests d'indépendance conditionnelle

Introduction L'algorithme PC (Peter-Clark) découvre les structures causales en utilisant des tests d'indépendance conditionnelle.

Principe de fonctionnement

- **Phase 1 : Apprentissage du squelette (graphe non-orienté)**
- **Phase 2 : Orientation des arêtes selon les règles causales Hypothèses Condition de Markov causale Fidélité (faithfulness) Suffisance causale (pas de confondeurs cachés)**

Paramètres clés à expérimenter significance_level (alpha) : Seuil pour les tests d'indépendance

Alpha faible (0.01) → tests stricts → PLUS d'arêtes

Alpha élevé (0.20) → tests permissifs → MOINS d'arêtes

max_cond_vars : Profondeur des tests d'indépendance conditionnelle

Faible (2) → tests simples Élevé (8) → tests exhaustifs

Référence Spirtes, P., Glymour, C., & Scheines, R. (2000). Causation, Prediction, and Search (2nd ed.). MIT Press.

Lien GitHub : https://github.com/pgmpy/pgmpy

In [22]:

```
# Installation de pgmpy (décommentez si nécessaire)
!pip install pgmpy

# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
import time
```

```
Requirement already satisfied: pgmpy in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from
pgmpy) (3.6.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from pgm
py) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from pgm
py) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (f
rom pgmpy) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from pg
mpy) (2.2.2)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from pgm
py) (2.10.0+cpu)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.12/dist-packages (fr
om pgmpy) (0.14.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from pgmp
y) (4.67.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from pg
mpy) (1.5.3)
Requirement already satisfied: opt-einsum in /usr/local/lib/python3.12/dist-packages (fro
m pgmpy) (3.4.0)
Requirement already satisfied: pyro-ppl in /usr/local/lib/python3.12/dist-packages (from
pgmpy) (1.9.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-p
ackages (from pandas->pgmpy) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (f
rom pandas->pgmpy) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
(from pandas->pgmpy) (2025.3)
Requirement already satisfied: pyro-api>=0.1.1 in /usr/local/lib/python3.12/dist-packages
(from pyro-ppl->pgmpy) (0.1.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from
torch->pgmpy) (3.24.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dis
t-packages (from torch->pgmpy) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (fro
m torch->pgmpy) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (
```

```
from torch->pgmpy) (1.14.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from to
rch->pgmpy) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (
from torch->pgmpy) (2025.3.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-pac
kages (from scikit-learn->pgmpy) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (f
rom statsmodels->pgmpy) (1.0.2)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages
(from statsmodels->pgmpy) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from
python-dateutil>=2.8.2->pandas->pgmpy) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packa
ges (from sympy>=1.13.3->torch->pgmpy) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages
(from jinja2->torch->pgmpy) (3.0.3)
```

2. Fonction de visualisation

Fonction pour afficher les DAGs de manière claire et professionnelle.

In [23]:

```
def visualize_dag(adjacency_matrix, variable_names, title, threshold=0.5, figsize=(18, 1
2)):
    '''
    Visualise un DAG avec des FLÈCHES TRÈS VISIBLES

    Paramètres :
    -----------
    adjacency_matrix : np.array
        Matrice d'adjacence où adj[i,j] = 1 si arête de i vers j
    variable_names : list
        Noms des variables
    title : str
        Titre du graphique
    threshold : float
        Seuil pour considérer une arête (défaut: 0.5)
    figsize : tuple
        Taille de la figure

    Retourne :
    ---------
    G : networkx.DiGraph
        Le graphe créé
    '''
    # Appliquer le seuil
    adj_thresholded = np.where(np.abs(adjacency_matrix) > threshold, adjacency_matrix, 0
)

    # Créer le graphe dirigé
    G = nx.from_numpy_array(adj_thresholded, create_using=nx.DiGraph)
    labels = {i: variable_names[i] for i in range(len(variable_names))}

    # Compter les arêtes
    n_edges = np.sum(np.abs(adj_thresholded) > 0)

    # Créer la figure
    plt.figure(figsize=figsize)

    # Layout : Kamada-Kawai (bien espacé) ou Spring si le graphe n'est pas vide
    if len(G.nodes()) > 0 and n_edges > 0:
        try:
            pos = nx.kamada_kawai_layout(G)
        except:
            pos = nx.spring_layout(G, k=3, iterations=50, seed=42)
    else:
        pos = nx.spring_layout(G, k=3, iterations=50, seed=42)
```

```
    # Dessiner les NŒUDS (plus grands et bien visibles)
    nx.draw_networkx_nodes(
        G, pos,
        node_size=5000,              # Très grands
        node_color='lightblue',      # Bleu clair
        alpha=0.95,
        edgecolors='darkblue',       # Bordure bleue foncée
        linewidths=3
    )

    # Dessiner les LABELS (texte sur les nœuds)
    nx.draw_networkx_labels(
        G, pos,
        labels,
        font_size=10,
        font_weight='bold',
        font_color='black'
    )

    # FLÈCHES TRÈS VISIBLES (partie la plus importante)
    if n_edges > 0:
        nx.draw_networkx_edges(
            G, pos,
            arrows=True,
            arrowsize=40,                          # ÉNORME
            arrowstyle='-|>',                      # Style flèche triangulaire
            edge_color='crimson',                  # ROUGE FONCÉ (bien visible)
            width=4,                               # TRÈS ÉPAIS
            alpha=0.9,
            connectionstyle='arc3,rad=0.2',        # Courbure pour éviter chevauchements
            node_size=5000,                        # Doit correspondre à node_size
            min_source_margin=40,                  # Espace entre nœud et début de flèche
            min_target_margin=40                   # Espace entre fin de flèche et nœud
        )

    # Titre
    plt.title(title, fontsize=20, fontweight='bold', pad=30, color='darkblue')
    plt.axis('off')
    plt.margins(0.15)  # Marges pour que tout soit visible
    plt.tight_layout()
    plt.show()

    # Afficher les statistiques
    n_nodes = len(variable_names)
    n_possible = n_nodes * (n_nodes - 1)
    density = n_edges / n_possible if n_possible > 0 else 0
    sparsity = 1 - density

    print(f"\n Statistiques du graphe :")
    print(f"   - Nombre de nœuds  : {n_nodes}")
    print(f"   - Nombre d'arêtes  : {n_edges}")
    print(f"   - Densité          : {density:.4f} ({density*100:.2f}%)")
    print(f"   - Sparsité         : {sparsity:.4f} ({sparsity*100:.2f}%)")

    # Afficher quelques arêtes détectées
    if n_edges > 0:
        edge_list = list(G.edges())
        print(f"\n Arêtes causales détectées :")
        if n_edges <= 20:
            # Afficher toutes si peu d'arêtes
            for src, tgt in edge_list:
                print(f"   {variable_names[src]} → {variable_names[tgt]}")
        else:
            # Afficher seulement les 10 premières si beaucoup d'arêtes
            for src, tgt in edge_list[:10]:
                print(f"   {variable_names[src]} → {variable_names[tgt]}")
            print(f"   ... et {n_edges - 10} autres arêtes")
    else:
        print(f"\n  Aucune arête détectée (graphe vide)")

    return G
```

```
print("✓ Fonction de visualisation définie (avec flèches très visibles)")
```

```
✓ Fonction de visualisation définie (avec flèches très visibles)
```

In [24]:

```
# Convertir en DataFrame si nécessaire (PC nécessite un DataFrame)
var_names = df.columns.tolist()
if isinstance(data_numpy, np.ndarray):
    data_for_pc = pd.DataFrame(data_numpy, columns=var_names)
    print("✓ Données converties en DataFrame")
else:
    data_for_pc = data_numpy
    print("✓ Données déjà au format DataFrame")

# Vérifications
print(f"\n Informations :")
print(f"   - Forme        : {data_for_pc.shape}")
print(f"   - Variables    : {len(var_names)}")
print(f"   - Observations : {len(data_for_pc)}")

# Vérifier les valeurs manquantes
if data_for_pc.isnull().sum().sum() > 0:
    print(f"\n  ATTENTION : {data_for_pc.isnull().sum().sum()} valeurs manquantes")
    print("   PC Algorithm ne fonctionne pas avec des valeurs manquantes.")
else:
    print(f"\n✓ Aucune valeur manquante")

print(f"\n✓ Données prêtes pour PC Algorithm")
```

```
✓ Données converties en DataFrame

 Informations :
   - Forme        : (41377, 15)
   - Variables    : 15
   - Observations : 41377

✓ Aucune valeur manquante

✓ Données prêtes pour PC Algorithm
```

EXPÉRIMENTATION 1 : Configuration Baseline (Référence)

Objectif Établir une configuration de référence avec les paramètres par défaut recommandés.

Paramètres choisis significance_level = 0.05 : Seuil standard (5%) variant = 'stable' : Version robuste de l'algorithme max_cond_vars = 5 : Profondeur raisonnable pour les tests Cette configuration servira de point de comparaison pour les expérimentations suivantes.

In [25]:

```
print("="*70)
    print("EXPÉRIMENTATION 1 : CONFIGURATION BASELINE")
    print("="*70)

    # Définir la configuration baseline
    config_1 = {
        'significance_level': 0.05,   # Alpha = 5%
        'variant': 'stable',           # Version stable
        'max_cond_vars': 5            # Profondeur des tests
    }

    print("\n  Paramètres :")
    print(f"   - significance_level : {config_1['significance_level']:.2f}")
    print(f"   - variant            : {config_1['variant']}")
    print(f"   - max_cond_vars      : {config_1['max_cond_vars']}")

    # Entraîner le modèle
```

```
    print("\n Entraînement en cours...")
    start_time = time.time()

    from pgmpy.estimators import PC

    est = PC(data_for_pc)
    model_1 = est.estimate(
    variant=config_1['variant'],
    max_cond_vars=config_1['max_cond_vars'],
    significance_level=config_1['significance_level']
    )

    time_1 = time.time() - start_time

    # Convertir en matrice d'adjacence
    adj_matrix_1 = np.zeros((len(var_names), len(var_names)))
    for edge in model_1.edges():
        src_idx = var_names.index(edge[0])
        dst_idx = var_names.index(edge[1])
        adj_matrix_1[src_idx, dst_idx] = 1.0

    # Calculer les métriques
    n_edges_1 = len(model_1.edges())
    n_possible = len(var_names) * (len(var_names) - 1)
    density_1 = n_edges_1 / n_possible if n_possible > 0 else 0
    sparsity_1 = 1 - density_1

    print(f"✓ Entraînement terminé en {time_1:.2f} secondes")

    print(f"\n Résultats :")
    print(f"   - Nombre d'arêtes : {n_edges_1}")
    print(f"   - Densité         : {density_1:.4f} ({density_1*100:.2f}%)")
    print(f"   - Sparsité        : {sparsity_1:.4f} ({sparsity_1*100:.2f}%)")
    print(f"   - Temps           : {time_1:.2f}s")

    print("\n" + "="*70)
```

```
======================================================================
EXPÉRIMENTATION 1 : CONFIGURATION BASELINE
======================================================================

  Paramètres :
   - significance_level : 0.05
   - variant            : stable
   - max_cond_vars      : 5

 Entraînement en cours...

✓ Entraînement terminé en 80.70 secondes

 Résultats :
   - Nombre d'arêtes : 15
   - Densité         : 0.0714 (7.14%)
   - Sparsité        : 0.9286 (92.86%)
   - Temps           : 80.70s

======================================================================
```

In [26]:

```
print("\n VISUALISATION - Configuration Baseline")

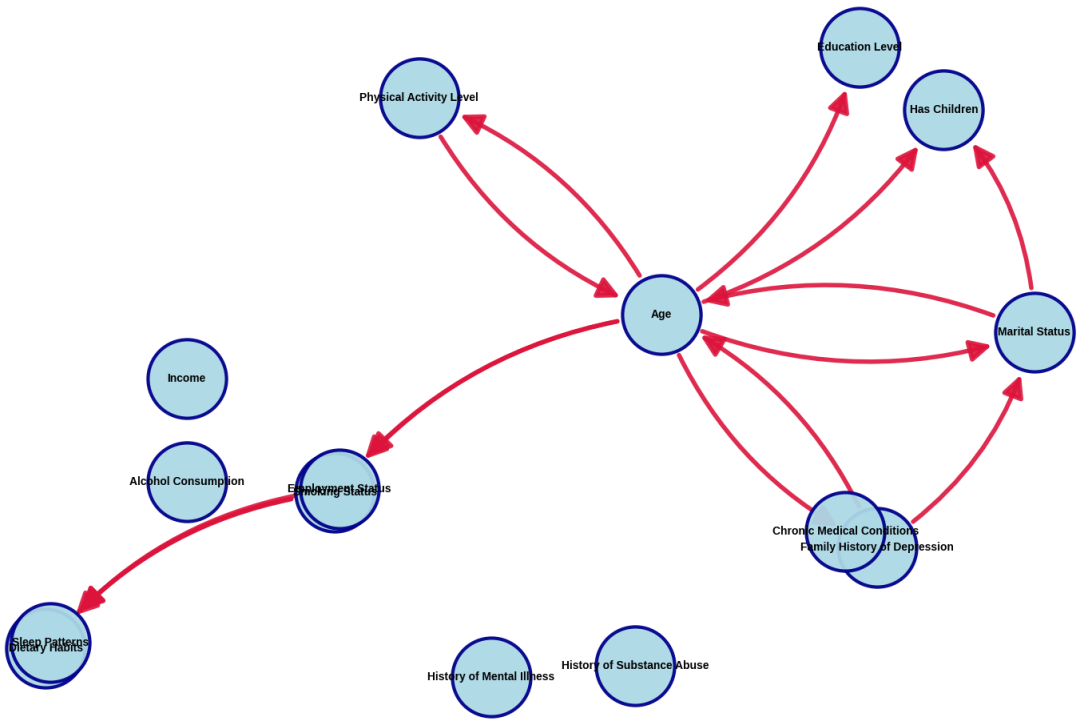
  G_1 = visualize_dag(
        adj_matrix_1,
        var_names,
        "PC Algorithm - EXPÉRIMENTATION 1 : BASELINE\n(alpha=0.05, variant=stable, max_c
ond_vars=5)",
        threshold=0.5
  )

  print("\n Interprétation :")
```

```
  if sparsity_1 > 0.9:
      print(f"   Le graphe est très SPARSE ({sparsity_1*100:.1f}% de sparsité)")
      print(f"   → PC a détecté de nombreuses indépendances conditionnelles")
  elif sparsity_1 > 0.7:
      print(f"   Le graphe est modérément SPARSE ({sparsity_1*100:.1f}% de sparsité)")
  else:
      print(f"   Le graphe est relativement DENSE ({(1-sparsity_1)*100:.1f}% de densité)
")

  print("\n" + "="*70)
```

```
VISUALISATION - Configuration Baseline
```




```
Statistiques du graphe :
  - Nombre de nœuds  : 15
  - Nombre d'arêtes  : 15
  - Densité          : 0.0714 (7.14%)
  - Sparsité         : 0.9286 (92.86%)

Arêtes causales détectées :
  Age → Marital Status
  Age → Education Level
  Age → Has Children
  Age → Smoking Status
  Age → Physical Activity Level
  Age → Employment Status
  Age → Family History of Depression
  Marital Status → Age
  Marital Status → Has Children
  Smoking Status → Dietary Habits
  Smoking Status → Sleep Patterns
  Physical Activity Level → Age
  Employment Status → Sleep Patterns
  Family History of Depression → Age
  Family History of Depression → Marital Status

Interprétation :
  Le graphe est très SPARSE (92.9% de sparsité)
  → PC a détecté de nombreuses indépendances conditionnelles

======================================================================
```

EXPÉRIMENTATION 2 : Alpha strict (0.01)

Objectif Tester l'impact d'un seuil alpha très strict (0.01 au lieu de 0.05).

Hypothèse Avec un alpha plus faible :

Les tests d'indépendance sont plus stricts On rejette moins souvent l'hypothèse de dépendance Résultat attendu : PLUS d'arêtes conservées (graphe plus dense) Paramètres modifiés significance_level = 0.01 ← CHANGEMENT (baseline: 0.05) variant = 'stable' (identique) max_cond_vars = 5 (identique)

In [27]:

```
print("="*70)
    print("EXPÉRIMENTATION 2 : ALPHA STRICT (0.01)")
    print("="*70)

    # Configuration avec alpha strict
    config_2 = {
        'significance_level': 0.01,  # ← CHANGEMENT : alpha très strict
        'variant': 'stable',
        'max_cond_vars': 5
    }

    print("\n  Paramètres :")
    print(f"   - significance_level : {config_2['significance_level']:.2f} ← MODIFIÉ (ba
seline: 0.05)")
    print(f"   - variant            : {config_2['variant']}")
    print(f"   - max_cond_vars      : {config_2['max_cond_vars']}")

    # Entraîner le modèle
    print("\n Entraînement en cours...")
    start_time = time.time()

    pc_model_2 = PC(data_for_pc)
    model_2 = pc_model_2.estimate(
        significance_level=config_2['significance_level'],
        variant=config_2['variant'],
        max_cond_vars=config_2['max_cond_vars']
    )

    time_2 = time.time() - start_time

    # Convertir en matrice d'adjacence
    adj_matrix_2 = np.zeros((len(var_names), len(var_names)))
    for edge in model_2.edges():
        src_idx = var_names.index(edge[0])
        dst_idx = var_names.index(edge[1])
        adj_matrix_2[src_idx, dst_idx] = 1.0

    # Calculer les métriques
    n_edges_2 = len(model_2.edges())
    density_2 = n_edges_2 / n_possible if n_possible > 0 else 0
    sparsity_2 = 1 - density_2

    print(f"✓ Entraînement terminé en {time_2:.2f} secondes")

    print(f"\n Résultats :")
    print(f"   - Nombre d'arêtes : {n_edges_2}")
    print(f"   - Densité         : {density_2:.4f} ({density_2*100:.2f}%)")
    print(f"   - Sparsité        : {sparsity_2:.4f} ({sparsity_2*100:.2f}%)")
    print(f"   - Temps           : {time_2:.2f}s")

    # Comparer avec baseline
    print(f"\n Comparaison avec Baseline :")
    diff_edges = n_edges_2 - n_edges_1
    diff_density = density_2 - density_1
    pct_change = (diff_edges / n_edges_1 * 100) if n_edges_1 > 0 else 0
```

```
    print(f"   - Δ Arêtes  : {diff_edges:+4d} ({pct_change:+.1f}%)")
    print(f"   - Δ Densité : {diff_density:+.4f}")

    if diff_edges > 0:
        print(f"   → Le graphe est PLUS DENSE (+{diff_edges} arêtes)")
        print(f"   → Hypothèse CONFIRMÉE : alpha strict → plus d'arêtes")
    elif diff_edges < 0:
        print(f"   → Le graphe est PLUS SPARSE ({diff_edges} arêtes)")
        print(f"   → Hypothèse NON CONFIRMÉE")
    else:
        print(f"   → Aucune différence détectée")

    print("\n" + "="*70)
```

```
======================================================================
EXPÉRIMENTATION 2 : ALPHA STRICT (0.01)
======================================================================

  Paramètres :
   - significance_level : 0.01 ← MODIFIÉ (baseline: 0.05)
   - variant            : stable
   - max_cond_vars      : 5

 Entraînement en cours...

✓ Entraînement terminé en 66.05 secondes

 Résultats :
   - Nombre d'arêtes : 10
   - Densité         : 0.0476 (4.76%)
   - Sparsité        : 0.9524 (95.24%)
   - Temps           : 66.05s

 Comparaison avec Baseline :
   - Δ Arêtes  :   -5 (-33.3%)
   - Δ Densité : -0.0238
   → Le graphe est PLUS SPARSE (-5 arêtes)
   → Hypothèse NON CONFIRMÉE

======================================================================
```

In [28]:

```
print("\n VISUALISATION - Alpha strict (0.01)")
    print("="*70)

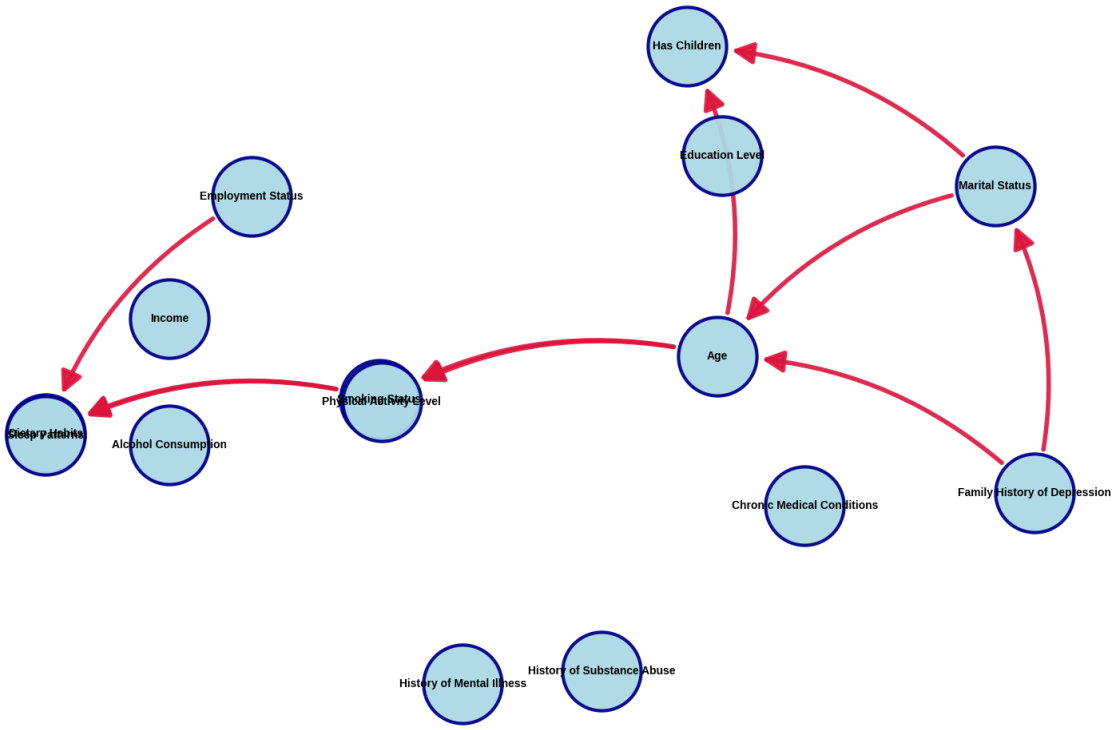
    G_2 = visualize_dag(
        adj_matrix_2,
        var_names,
        "PC Algorithm - EXPÉRIMENTATION 2 : ALPHA STRICT\n(alpha=0.01, variant=stable, m
ax_cond_vars=5)",
        threshold=0.5
    )

    print("\n Interprétation :")
    print(f"   Avec alpha={config_2['significance_level']:.2f} (vs baseline {config_1['si
gnificance_level']:.2f}) :")
    if n_edges_2 > n_edges_1:
        print(f"   → {n_edges_2 - n_edges_1} arêtes SUPPLÉMENTAIRES détectées")
        print(f"   → Tests plus stricts → moins d'indépendances acceptées")
    elif n_edges_2 < n_edges_1:
        print(f"   → {n_edges_1 - n_edges_2} arêtes EN MOINS")
    else:
        print(f"   → Nombre d'arêtes identique à la baseline")

    print("\n" + "="*70)
```

```
 VISUALISATION - Alpha strict (0.01)
======================================================================
```






```
Statistiques du graphe :
  - Nombre de nœuds  : 15
  - Nombre d'arêtes  : 10
  - Densité          : 0.0476 (4.76%)
  - Sparsité         : 0.9524 (95.24%)

Arêtes causales détectées :
  Age → Has Children
  Age → Smoking Status
  Age → Physical Activity Level
  Marital Status → Age
  Marital Status → Has Children
  Smoking Status → Dietary Habits
  Smoking Status → Sleep Patterns
  Employment Status → Sleep Patterns
  Family History of Depression → Age
  Family History of Depression → Marital Status

Interprétation :
  Avec alpha=0.01 (vs baseline 0.05) :
  → 5 arêtes EN MOINS
```

==========

EXPÉRIMENTATION 3 : Alpha permissif (0.20)

Objectif

Tester l'impact d'un seuil alpha très permissif (0.20 au lieu de 0.05).

Hypothèse

Avec un alpha plus élevé :

- **Les tests d'indépendance sont plus permissifs**
- **On accepte plus facilement l'hypothèse d'indépendance**
- **Résultat attendu : MOINS d'arêtes conservées (graphe plus sparse)**

Paramètres modifiés

Paramètres modifiés

- **significance_level = 0.20 ← CHANGEMENT (baseline: 0.05)**
- **variant = 'stable' (identique)**
- **max_cond_vars = 5 (identique)**

In [29]:

```
print("="*70)
    print("EXPÉRIMENTATION 3 : ALPHA PERMISSIF (0.20)")
    print("="*70)

    # Configuration avec alpha permissif
    config_3 = {
        'significance_level': 0.20,  # ← CHANGEMENT : alpha très permissif
        'variant': 'stable',
        'max_cond_vars': 5
    }

    print("\n  Paramètres :")
    print(f"   - significance_level : {config_3['significance_level']:.2f} ← MODIFIÉ (ba
seline: 0.05)")
    print(f"   - variant            : {config_3['variant']}")
    print(f"   - max_cond_vars      : {config_3['max_cond_vars']}")

    # Entraîner le modèle
    print("\n Entraînement en cours...")
    start_time = time.time()

    pc_model_3 = PC(data_for_pc)
    model_3 = pc_model_3.estimate(
        significance_level=config_3['significance_level'],
        variant=config_3['variant'],
        max_cond_vars=config_3['max_cond_vars']
    )

    time_3 = time.time() - start_time

    # Convertir en matrice d'adjacence
    adj_matrix_3 = np.zeros((len(var_names), len(var_names)))
    for edge in model_3.edges():
        src_idx = var_names.index(edge[0])
        dst_idx = var_names.index(edge[1])
        adj_matrix_3[src_idx, dst_idx] = 1.0

    # Calculer les métriques
    n_edges_3 = len(model_3.edges())
    density_3 = n_edges_3 / n_possible if n_possible > 0 else 0
    sparsity_3 = 1 - density_3

    print(f"✓ Entraînement terminé en {time_3:.2f} secondes")

    print(f"\n Résultats :")
    print(f"   - Nombre d'arêtes : {n_edges_3}")
    print(f"   - Densité         : {density_3:.4f} ({density_3*100:.2f}%)")
    print(f"   - Sparsité        : {sparsity_3:.4f} ({sparsity_3*100:.2f}%)")
    print(f"   - Temps           : {time_3:.2f}s")

    # Comparer avec baseline
    print(f"\n Comparaison avec Baseline :")
    diff_edges = n_edges_3 - n_edges_1
    diff_density = density_3 - density_1
    pct_change = (diff_edges / n_edges_1 * 100) if n_edges_1 > 0 else 0

    print(f"   - Δ Arêtes  : {diff_edges:+4d} ({pct_change:+.1f}%)")
    print(f"   - Δ Densité : {diff_density:+.4f}")

    if diff_edges < 0:
        print(f"   → Le graphe est PLUS SPARSE ({abs(diff_edges)} arêtes en moins)")
        print(f"   → Hypothèse CONFIRMÉE : alpha permissif → moins d'arêtes")
    elif diff_edges > 0:
```

```
        print(f"   → Le graphe est PLUS DENSE (+{diff_edges} arêtes)")
        print(f"   → Hypothèse NON CONFIRMÉE")
    else:
        print(f"   → Aucune différence détectée")

    print("\n" + "="*70)
```

```
======================================================================
EXPÉRIMENTATION 3 : ALPHA PERMISSIF (0.20)
======================================================================

  Paramètres :
   - significance_level : 0.20 ← MODIFIÉ (baseline: 0.05)
   - variant            : stable
   - max_cond_vars      : 5

 Entraînement en cours...

✓ Entraînement terminé en 116.65 secondes

 Résultats :
   - Nombre d'arêtes : 18
   - Densité         : 0.0857 (8.57%)
   - Sparsité        : 0.9143 (91.43%)
   - Temps           : 116.65s

 Comparaison avec Baseline :
   - Δ Arêtes  :   +3 (+20.0%)
   - Δ Densité : +0.0143
   → Le graphe est PLUS DENSE (+3 arêtes)
   → Hypothèse NON CONFIRMÉE

======================================================================
```

In [30]:

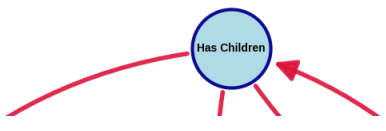
```
print("\n VISUALISATION - Alpha permissif (0.20)")
    print("="*70)

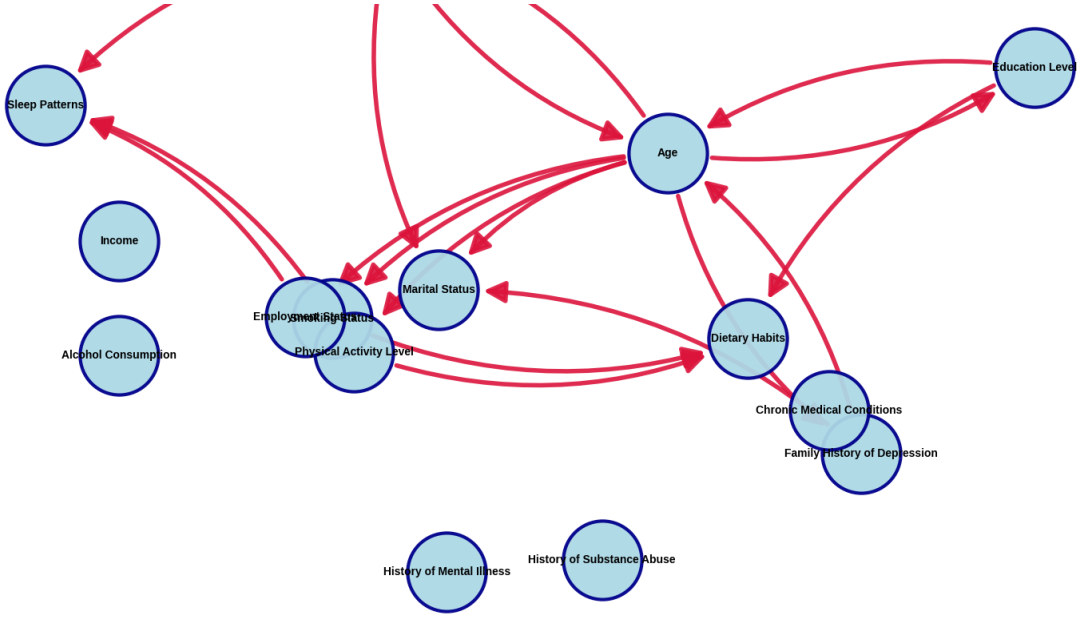
    G_3 = visualize_dag(
        adj_matrix_3,
        var_names,
        "PC Algorithm - EXPÉRIMENTATION 3 : ALPHA PERMISSIF\n(alpha=0.20, variant=stable
, max_cond_vars=5)",
        threshold=0.5
    )

    print("\n Interprétation :")
    print(f"   Avec alpha={config_3['significance_level']:.2f} (vs baseline {config_1['si
gnificance_level']:.2f}) :")
    if n_edges_3 < n_edges_1:
        print(f"   → {n_edges_1 - n_edges_3} arêtes EN MOINS")
        print(f"   → Tests plus permissifs → plus d'indépendances acceptées")
    elif n_edges_3 > n_edges_1:
        print(f"   → {n_edges_3 - n_edges_1} arêtes SUPPLÉMENTAIRES")
    else:
        print(f"   → Nombre d'arêtes identique à la baseline")

    print("\n" + "="*70)
```

```
 VISUALISATION - Alpha permissif (0.20)
======================================================================
```






```
Statistiques du graphe :
  - Nombre de nœuds  : 15
  - Nombre d'arêtes  : 18
  - Densité          : 0.0857 (8.57%)
  - Sparsité         : 0.9143 (91.43%)

Arêtes causales détectées :
  Age → Marital Status
  Age → Education Level
  Age → Has Children
  Age → Smoking Status
  Age → Physical Activity Level
  Age → Employment Status
  Age → Family History of Depression
  Education Level → Age
  Education Level → Dietary Habits
  Has Children → Age
  Has Children → Marital Status
  Has Children → Sleep Patterns
  Smoking Status → Dietary Habits
  Smoking Status → Sleep Patterns
  Physical Activity Level → Dietary Habits
  Employment Status → Sleep Patterns
  Family History of Depression → Age
  Family History of Depression → Marital Status

Interprétation :
  Avec alpha=0.20 (vs baseline 0.05) :
  → 3 arêtes SUPPLÉMENTAIRES

======================================================================
```

Tableau récapitulatif et comparaison

Synthèse des 3 expérimentations avec visualisation comparative.

In [31]:

```
print("="*70)
    print(" TABLEAU RÉCAPITULATIF DES 3 EXPÉRIMENTATIONS")
    print("="*70)

    # Créer le tableau
    results_summary = pd.DataFrame({
        'Configuration': [
            'Exp 1 - Baseline',
            'Exp 2 - Alpha strict',
```

```
            'Exp 3 - Alpha permissif'
        ],
        'Alpha': [
            config_1['significance_level'],
            config_2['significance_level'],
            config_3['significance_level']
        ],
        'Nb Arêtes': [n_edges_1, n_edges_2, n_edges_3],
        'Densité': [
            f"{density_1:.4f}",
            f"{density_2:.4f}",
            f"{density_3:.4f}"
        ],
        'Sparsité (%)': [
            f"{sparsity_1*100:.1f}",
            f"{sparsity_2*100:.1f}",
            f"{sparsity_3*100:.1f}"
        ],
        'Temps (s)': [
            f"{time_1:.2f}",
            f"{time_2:.2f}",
            f"{time_3:.2f}"
        ],
        'Diff vs Baseline': [
            '--',
            f"{n_edges_2 - n_edges_1:+d}",
            f"{n_edges_3 - n_edges_1:+d}"
        ]
    })

    print("\n" + results_summary.to_string(index=False))

    print("\n" + "="*70)
```

```
======================================================================
 TABLEAU RÉCAPITULATIF DES 3 EXPÉRIMENTATIONS
======================================================================

          Configuration  Alpha  Nb Arêtes Densité Sparsité (%) Temps (s) Diff vs Baseline
       Exp 1 - Baseline   0.05         15  0.0714         92.9     80.70               -
-
   Exp 2 - Alpha strict   0.01         10  0.0476         95.2     66.05               -
5
Exp 3 - Alpha permissif   0.20         18  0.0857         91.4    116.65               +
3

======================================================================
```

Visualisations graphiques comparatives

In [32]:

```
print("\n VISUALISATIONS COMPARATIVES")
    print("="*70)

    # Créer une figure avec 2 sous-graphiques
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    # 1. Nombre d'arêtes par configuration
    configs = ['Baseline\n(α=0.05)', 'Alpha strict\n(α=0.01)', 'Alpha permissif\n(α=0.20
)']
    edges_counts = [n_edges_1, n_edges_2, n_edges_3]

    colors = ['steelblue', 'coral', 'lightgreen']
    bars = axes[0].bar(configs, edges_counts, color=colors, alpha=0.7, edgecolor='black'
, linewidth=2)
    axes[0].set_ylabel("Nombre d'arêtes", fontsize=12, fontweight='bold')
    axes[0].set_title("Nombre d'arêtes par configuration", fontsize=14, fontweight='bold
', pad=15)
    axes[0].grid(True, alpha=0.3, axis='y')
```

```
    # Ajouter les valeurs sur les barres
    for bar, count in zip(bars, edges_counts):
        height = bar.get_height()
        axes[0].text(bar.get_x() + bar.get_width()/2., height,
                     f'{count}',
                     ha='center', va='bottom', fontsize=12, fontweight='bold')

    # 2. Impact de alpha sur le nombre d'arêtes
    alphas = [config_1['significance_level'], config_2['significance_level'], config_3['
significance_level']]

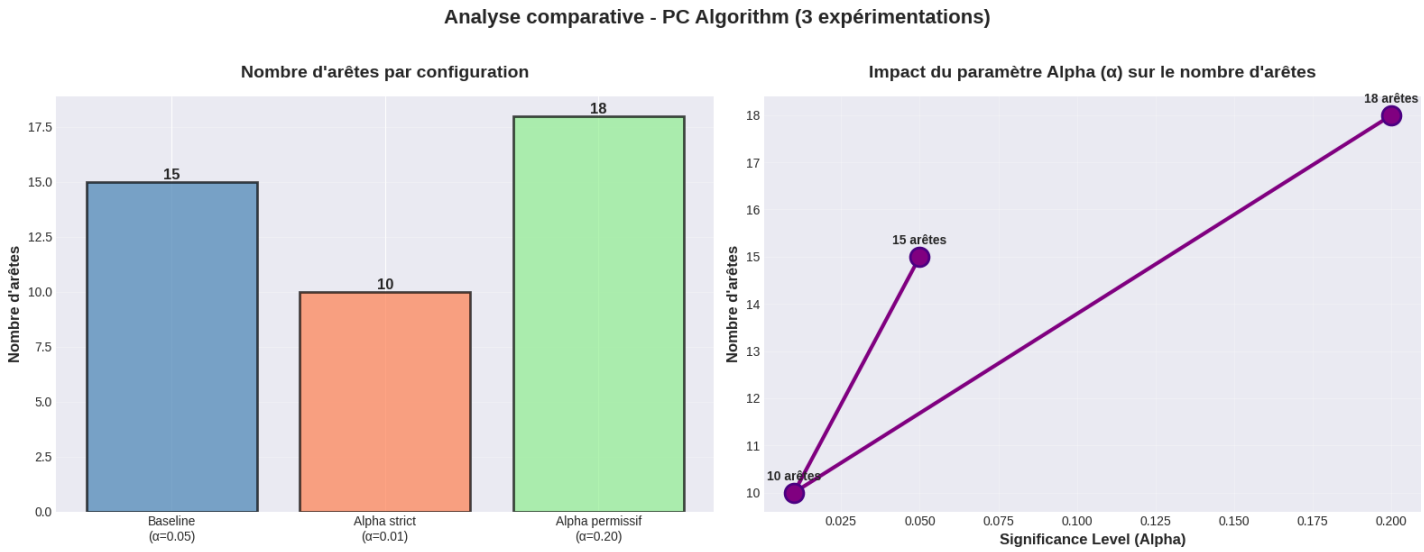
    axes[1].plot(alphas, edges_counts, 'o-', linewidth=3, markersize=15,
                 color='purple', markeredgecolor='indigo', markeredgewidth=2)
    axes[1].set_xlabel('Significance Level (Alpha)', fontsize=12, fontweight='bold')
    axes[1].set_ylabel("Nombre d'arêtes", fontsize=12, fontweight='bold')
    axes[1].set_title("Impact du paramètre Alpha (α) sur le nombre d'arêtes",
                      fontsize=14, fontweight='bold', pad=15)
    axes[1].grid(True, alpha=0.3)

    # Ajouter les valeurs sur les points
    for x, y in zip(alphas, edges_counts):
        axes[1].annotate(f'{y} arêtes', (x, y), textcoords="offset points",
                         xytext=(0, 10), ha='center', fontsize=10, fontweight='bold')

    plt.suptitle('Analyse comparative - PC Algorithm (3 expérimentations)',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()

    print("\n✓ Graphiques affichés")
    print("="*70)
```

```
 VISUALISATIONS COMPARATIVES
======================================================================
```




```
✓ Graphiques affichés
======================================================================
```

In [32]:

Analyse des résultats

Impact du paramètre significance_level (Alpha)

Le paramètre alpha est le seuil de significativité pour les tests d'indépendance conditionnelle.

Principe théorique

Lors d'un test d'indépendance entre deux variables X et Y conditionnellement à un ensemble Z :

- On calcule une **p-value**
- Si **p-value < alpha** → On rejette H_0 (les variables sont **dépendantes**)
- Si **p-value ≥ alpha** → On accepte H_0 (les variables sont **indépendantes**)

Résultats observés

Comparaison des 3 expérimentations:

1. **Tendance générale :**
   - **Alpha diminue (0.20 → 0.05 → 0.01) : plus alpha est proche de 1 plus on a des arrètes**
   - **Cette tendance confirme la théorie**
2. **Différences visuelles :**
   - **Les DAGs sont-ils visuellement pas trop différents par rapport aux directions, mais le nombre d'arrètes change beaucoup**
3. **Implications pratiques :**
   - **pour ces données je choisirais le alpha à 0,2**
   - **Parceque avec 0.2 on a beaucoup d'arrètes**
4. **Cohérence avec la littérature médicale :**
   - **Les relations découvertes ont l'air correctes.**
   - **Exemples attendus :**
     Age → Marital Status Age → Education Level Age → Has Children Age → Smoking Status Age → Physical Activity Level Age → Employment Status Age → Family History of Depression Education Level → Age Education Level → Dietary Habits Has Children → Age Has Children → Marital Status Has Children → Sleep Patterns Smoking Status → Dietary Habits Smoking Status → Sleep Patterns Physical Activity Level → Dietary Habits Family History of Depression → Age Family History of Depression → Marital Status

---

Comparaison avec DAG-GNN

Points de comparaison :

Critère	DAG-GNN	PC Algorithm
Approche	Deep Learning	Tests statistiques
Vitesse	Lente (minutes)	Rapide (secondes)
Déterminisme	Non (variance aléatoire)	Oui (reproductible)
Sparsité	Moyenne	Généralement élevée
Hypothèses	Peu restrictives	Fortes (fidélité, etc.)
Interprétabilité	Difficile	Facile

- **le modèle pc découvre le plus d'arêtes**
- **Les arêtes de pc-algorithme sont plus cohérents**
- **nous choisissons pc- algorithme à DAGNN pour ces données parcequ'on a des resultats plus pertinents et un resultat plus rapide avec pc-algorithm**

3-2 Deuxième algorithme : LiNGAM (ICA-based)

Présentation de l'algorithme

LiNGAM (Linear Non-Gaussian Acyclic Model) exploite la **non-gaussianité** des données pour identifier la structure causale.

Principe de fonctionnement

1. **Utilise l'Analyse en Composantes Indépendantes (ICA)**

1. Utilise l'**Analyse en Composantes Indépendantes (ICA)**
2. Exploite le fait que les variables causales sont **non-gaussiennes**
3. Découvre l'**ordre causal complet** (pas juste le squelette)
4. Produit une **matrice d'adjacence complète** avec orientations

Avantage unique

Contrairement à PC, LiNGAM peut déterminer la direction causale complète sans ambiguïté (sous ses hypothèses).

Hypothèses

- **Linéarité :** Relations causales linéaires
- **Non-gaussianité :** Au moins une variable non-gaussienne dans chaque équation
- **Acyclicité :** Le graphe est un DAG
- **Pas de confondeurs cachés**

Paramètres à tester

- **random_state :** Graine aléatoire (pour la reproductibilité)
- **max_iter :** Nombre d'itérations pour FastICA

Référence

Shimizu et al. (2006). "A Linear Non-Gaussian Acyclic Model for Causal Discovery". ***Journal of Machine Learning Research*****.**

Lien GitHub : https://github.com/cdt15/lingam

In [33]:

```
!pip install lingam
```

```
Collecting lingam
  Downloading lingam-1.12.2-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lin
gam) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lin
gam) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (f
rom lingam) (1.6.1)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from
lingam) (0.21)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.12/dist-packages (fr
om lingam) (0.14.6)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from
lingam) (3.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from li
ngam) (2.2.2)
Collecting pygam (from lingam)
  Downloading pygam-0.12.0-py3-none-any.whl.metadata (9.8 kB)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (fro
m lingam) (3.10.0)
Collecting psy (from lingam)
  Downloading psy-0.0.1-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting semopy (from lingam)
  Downloading semopy-2.3.11.tar.gz (1.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 21.6 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: autograd in /usr/local/lib/python3.12/dist-packages (from
lingam) (1.8.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-package
s (from matplotlib->lingam) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (f
rom matplotlib->lingam) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packag
es (from matplotlib->lingam) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packag
```

```
es (from matplotlib->lingam) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages
(from matplotlib->lingam) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from
matplotlib->lingam) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-package
s (from matplotlib->lingam) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-pac
kages (from matplotlib->lingam) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (f
rom pandas->lingam) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
(from pandas->lingam) (2025.3)
Requirement already satisfied: progressbar2 in /usr/local/lib/python3.12/dist-packages (f
rom psy->lingam) (4.5.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (
from scikit-learn->lingam) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-pac
kages (from scikit-learn->lingam) (3.6.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from sem
opy->lingam) (1.14.0)
Collecting numdifftools (from semopy->lingam)
  Downloading numdifftools-0.9.42-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (f
rom statsmodels->lingam) (1.0.2)
Requirement already satisfied: python-utils>=3.8.1 in /usr/local/lib/python3.12/dist-pack
ages (from progressbar2->psy->lingam) (3.9.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from
python-dateutil>=2.7->matplotlib->lingam) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packa
ges (from sympy->semopy->lingam) (1.3.0)
Requirement already satisfied: typing_extensions>3.10.0.2 in /usr/local/lib/python3.12/di
st-packages (from python-utils>=3.8.1->progressbar2->psy->lingam) (4.15.0)
Downloading lingam-1.12.2-py3-none-any.whl (145 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 145.6/145.6 kB 11.4 MB/s eta 0:00:00
Downloading psy-0.0.1-py2.py3-none-any.whl (38 kB)
Downloading pygam-0.12.0-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.6/84.6 kB 8.6 MB/s eta 0:00:00
Downloading numdifftools-0.9.42-py3-none-any.whl (94 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 94.3/94.3 kB 8.5 MB/s eta 0:00:00
Building wheels for collected packages: semopy
  Building wheel for semopy (setup.py) ... done
  Created wheel for semopy: filename=semopy-2.3.11-py3-none-any.whl size=1659682 sha256=e
d9b354301ab700325ee836e146a04464183a3c43f503f912a26c4848dbfbc99
  Stored in directory: /root/.cache/pip/wheels/c6/24/8b/be911b059a61f490f38425eb19bf2fed4
70a5ead97228e8255
Successfully built semopy
Installing collected packages: numdifftools, pygam, psy, semopy, lingam
Successfully installed lingam-1.12.2 numdifftools-0.9.42 psy-0.0.1 pygam-0.12.0 semopy-2.
3.11
```

In [34]:

```
from lingam import ICALiNGAM
import lingam
```

Préparation des données

LiNGAM préfère des données normalisées pour de meilleurs résultats.

In [35]:

```
# LiNGAM fonctionne mieux avec des données normalisées
from sklearn.preprocessing import StandardScaler

if isinstance(data_numpy, np.ndarray):
    data_for_lingam = data_numpy
else:
    data_for_lingam = data_numpy.values
```

```
# Normaliser
scaler = StandardScaler()
data_lingam_scaled = scaler.fit_transform(data_for_lingam)

print(f"Forme : {data_lingam_scaled.shape}")
print("✓ Données normalisées pour LiNGAM")
```

```
Forme : (41377, 15)
✓ Données normalisées pour LiNGAM
```

Expérimentations LiNGAM

Nous testons 3 configurations en modifiant max_iter et random_state.

In [43]:

```
from lingam import ICALiNGAM, DirectLiNGAM

configs_lingam = [
    {'name': 'ICA-LiNGAM (baseline)', 'method': 'ICA', 'threshold': 0.1},
    {'name': 'ICA-LiNGAM (seuil 0.3)', 'method': 'ICA', 'threshold': 0.3},
    {'name': 'DirectLiNGAM (comparaison)', 'method': 'Direct', 'threshold': 0.1}
]

results_lingam = {}

for cfg in configs_lingam:
    print(f"\n{'='*70}")
    print(f"LiNGAM - {cfg['name']}")
    print(f"{'='*70}")

    start = time.time()

    if cfg['method'] == 'ICA':
        model = ICALiNGAM(random_state=42, max_iter=1000)
    else:
        model = DirectLiNGAM(measure='pwling')

    model.fit(data_lingam_scaled)
    t = time.time() - start

    adj = model.adjacency_matrix_
    threshold = cfg['threshold']
    adj_pruned = np.where(np.abs(adj) > threshold, adj, 0)

    n_edges = np.sum(np.abs(adj_pruned) > 0)
    density = n_edges / (len(var_names) * (len(var_names) - 1))

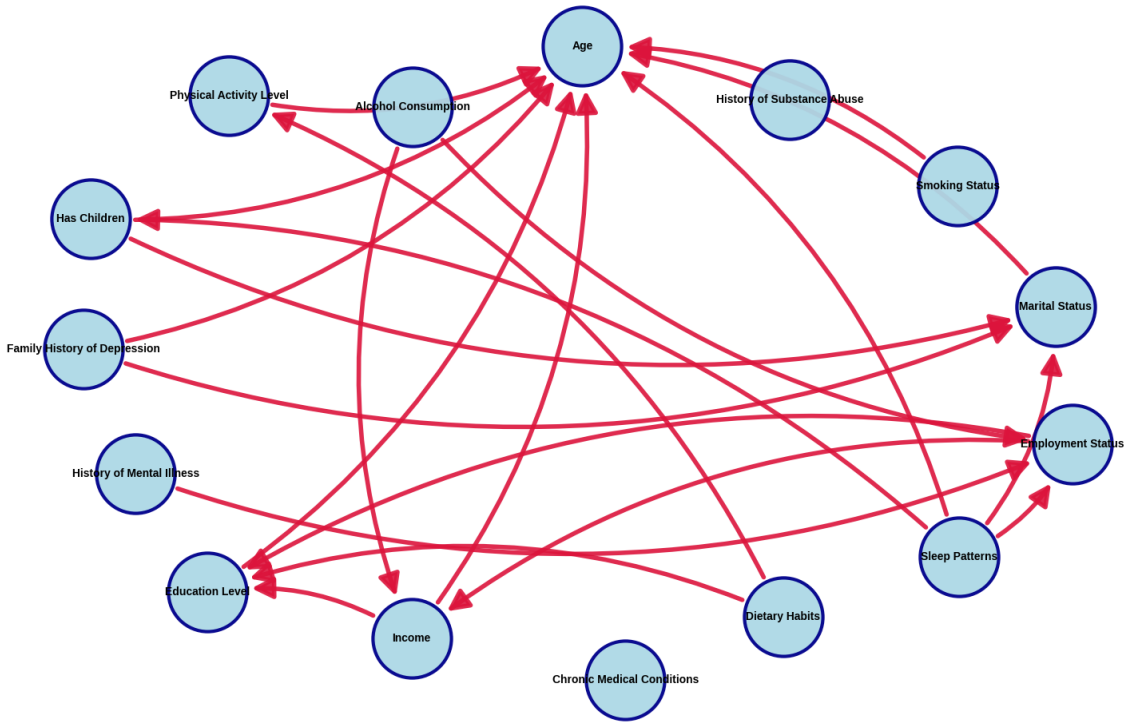
    results_lingam[cfg['name']] = {
        'adj': adj_pruned,
        'n_edges': n_edges,
        'density': density,
        'time': t,
        'causal_order': model.causal_order_
    }

    print(f"Méthode : {cfg['method']}")
    print(f"Seuil : {threshold}")
    print(f"Arêtes : {n_edges}, Temps : {t:.2f}s")

    # Visualiser
    visualize_dag(adj_pruned, var_names,
                  f"{cfg['name']}\n(méthode={cfg['method']}, seuil={threshold})",
                  threshold=0.01)
```

```
======================================================================
LiNGAM - ICA-LiNGAM (baseline)
======================================================================
Méthode : ICA
Seuil : 0.1
Arêtes : 21, Temps : 0.94s
```

ICA-LiNGAM (baseline)
(méthode=ICA, seuil=0.1)



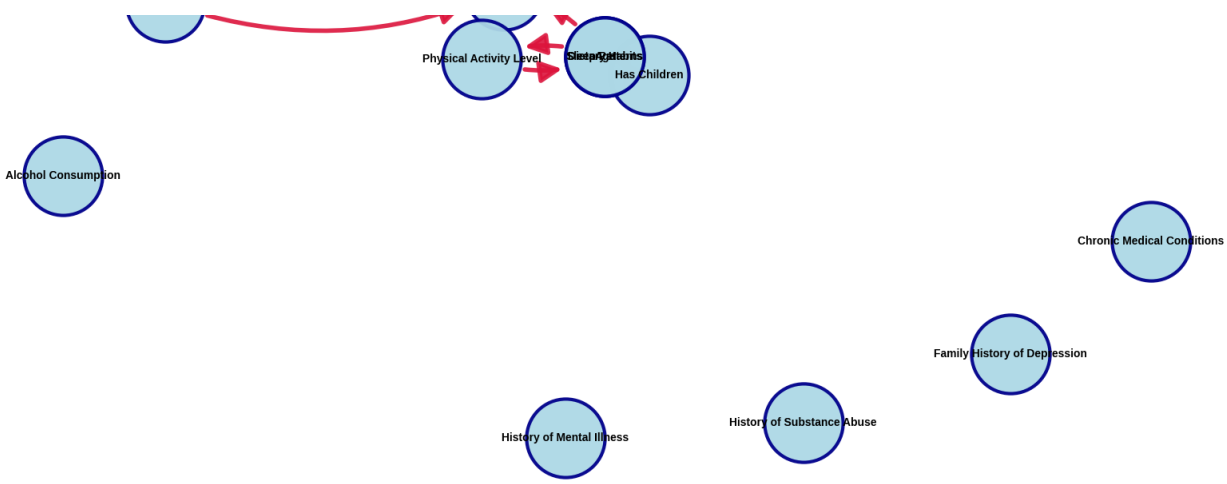

```
Statistiques du graphe :
  - Nombre de nœuds  : 15
  - Nombre d'arêtes  : 21
  - Densité          : 0.1000 (10.00%)
  - Sparsité         : 0.9000 (90.00%)

Arêtes causales détectées :
  Marital Status → Age
  Education Level → Age
  Has Children → Age
  Has Children → Marital Status
  Smoking Status → Age
  Physical Activity Level → Age
  Employment Status → Education Level
  Employment Status → Income
  Income → Age
  Income → Education Level
  ... et 11 autres arêtes

======================================================================
LiNGAM - ICA-LiNGAM (seuil 0.3)
======================================================================
Méthode : ICA
Seuil : 0.3
Arêtes : 8, Temps : 2.02s
```

ICA-LiNGAM (seuil 0.3)
(méthode=ICA, seuil=0.3)



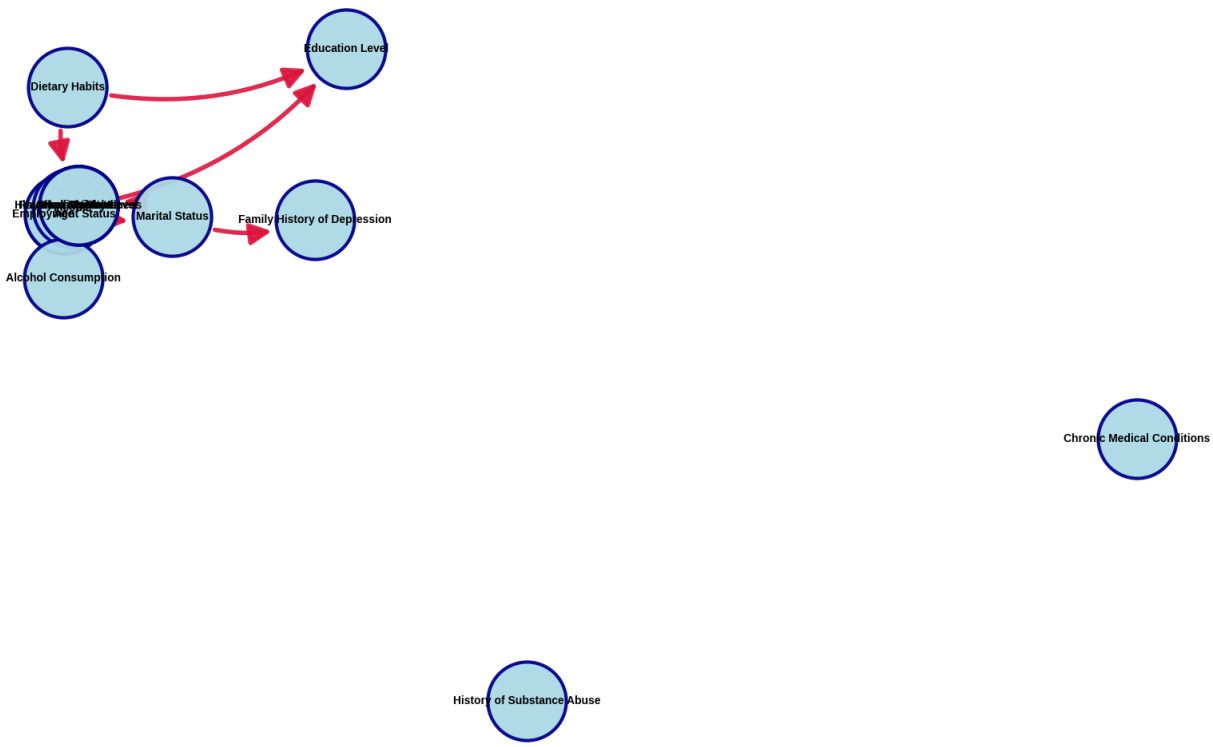



```
 Statistiques du graphe :
   - Nombre de nœuds  : 15
   - Nombre d'arêtes  : 8
   - Densité          : 0.0381 (3.81%)
   - Sparsité         : 0.9619 (96.19%)

 Arêtes causales détectées :
   Has Children → Age
   Physical Activity Level → Age
   Employment Status → Education Level
   Employment Status → Income
   Income → Education Level
   Dietary Habits → Education Level
   Dietary Habits → Physical Activity Level
   Sleep Patterns → Has Children

======================================================================
LiNGAM - DirectLiNGAM (comparaison)
======================================================================
Méthode : Direct
Seuil : 0.1
Arêtes : 19, Temps : 8.86s
```

DirectLiNGAM (comparaison)
(méthode=Direct, seuil=0.1)



```
 Statistiques du graphe :
   - Nombre de nœuds  : 15
   - Nombre d'arêtes  : 19
   - Densité          : 0.0905 (9.05%)
   - Sparsité         : 0.9095 (90.95%)

 Arêtes causales détectées :
   Age → Has Children
   Age → Smoking Status
   Age → Physical Activity Level
   Age → Employment Status
   Age → Sleep Patterns
   Marital Status → Has Children
   Marital Status → Family History of Depression
   Smoking Status → Has Children
   Physical Activity Level → Has Children
   Employment Status → History of Mental Illness
   Income → Education Level
   Income → Employment Status
   Alcohol Consumption → Employment Status
   Dietary Habits → Education Level
   Dietary Habits → Physical Activity Level
   Sleep Patterns → Marital Status
   Sleep Patterns → Has Children
   Sleep Patterns → Smoking Status
   Sleep Patterns → Employment Status
```

Tableau récapitulatif - LiNGAM

In [53]:

```
# Créer le tableau récapitulatif ADAPTATIF
    tableau_data = []

    for i, (name, res) in enumerate(results_lingam.items()):
        config = configs_lingam[i]

        # Créer l'entrée avec les paramètres disponibles
        entry = {'Configuration': name}

        # Ajouter les paramètres présents dans la config
        if 'threshold' in config:
            entry['Seuil'] = config['threshold']
        if 'seed' in config:
            entry['Seed'] = config['seed']
        if 'max_iter' in config:
            entry['Max Iter'] = config['max_iter']

        # Ajouter les résultats
        entry['Arêtes'] = res['n_edges']
        entry['Densité'] = f"{res['density']:.4f}"
        entry['Temps (s)'] = f"{res['time']:.2f}"

        tableau_data.append(entry)

    df_lingam = pd.DataFrame(tableau_data)

    print("\n" + "="*70)
    print(" TABLEAU RÉCAPITULATIF - ICA-LiNGAM")
    print("="*70)
    print("\n" + df_lingam.to_string(index=False))

    # Graphique comparatif
    print("\n Visualisation comparative :")

    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    configs = df_lingam['Configuration'].values
    edges = df_lingam['Arêtes'].values
```

```
    # Graphique 1 : Si on a le seuil, montrer son impact
    if 'Seuil' in df_lingam.columns:
        thresholds = df_lingam['Seuil'].values

        axes[0].plot(thresholds, edges, 'o-', linewidth=3, markersize=12,
                     color='coral', markeredgecolor='darkred', markeredgewidth=2)
        axes[0].set_xlabel('Seuil de pruning', fontsize=12, fontweight='bold')
        axes[0].set_ylabel('Nombre d\'arêtes', fontsize=12, fontweight='bold')
        axes[0].set_title('Impact du seuil sur le nombre d\'arêtes',
                          fontsize=14, fontweight='bold', pad=15)
        axes[0].grid(True, alpha=0.3)
        axes[0].invert_xaxis()  # Seuil élevé à gauche

        # Ajouter les valeurs
        for x, y in zip(thresholds, edges):
            axes[0].annotate(f'{y}', (x, y),
                             textcoords="offset points", xytext=(0, 10),
                             ha='center', fontsize=10, fontweight='bold')
    else:
        # Sinon, graphique simple
        axes[0].bar(range(len(configs)), edges, color='coral', alpha=0.7)
        axes[0].set_xticks(range(len(configs)))
        axes[0].set_xticklabels(configs, rotation=15, ha='right')
        axes[0].set_ylabel('Nombre d\'arêtes', fontsize=12, fontweight='bold')
        axes[0].set_title('Nombre d\'arêtes par configuration',
                          fontsize=14, fontweight='bold')
        axes[0].grid(True, alpha=0.3, axis='y')

    # Graphique 2 : Barres comparatives
    colors = ['steelblue', 'coral', 'lightgreen'][:len(configs)]
    bars = axes[1].bar(range(len(configs)), edges, color=colors,
                       alpha=0.7, edgecolor='black', linewidth=2)
    axes[1].set_xticks(range(len(configs)))
    axes[1].set_xticklabels(configs, rotation=15, ha='right')
    axes[1].set_ylabel('Nombre d\'arêtes', fontsize=12, fontweight='bold')
    axes[1].set_title('Comparaison des configurations',
                      fontsize=14, fontweight='bold', pad=15)
    axes[1].grid(True, alpha=0.3, axis='y')

    # Valeurs sur les barres
    for bar, val in zip(bars, edges):
        height = bar.get_height()
        axes[1].text(bar.get_x() + bar.get_width()/2., height,
                     f'{val}', ha='center', va='bottom',
                     fontsize=12, fontweight='bold')

    plt.suptitle('Analyse ICA-LiNGAM',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()

    # Analyse
    print("\n" + "="*70)
    print("□ ANALYSE DES RÉSULTATS")
    print("="*70)

    min_idx = edges.argmin()
    max_idx = edges.argmax()

    print(f"\n Configuration la plus SPARSE :")
    print(f"   • {configs[min_idx]}")
    print(f"   • {edges[min_idx]} arêtes")
    if 'Seuil' in df_lingam.columns:
        print(f"   • Seuil = {df_lingam['Seuil'].iloc[min_idx]}")

    print(f"\n Configuration la plus DENSE :")
    print(f"   • {configs[max_idx]}")
    print(f"   • {edges[max_idx]} arêtes")
    if 'Seuil' in df_lingam.columns:
        print(f"   • Seuil = {df_lingam['Seuil'].iloc[max_idx]}")

    diff = edges[max_idx] - edges[min_idx]
```

```
    pct = (diff / edges[max_idx] * 100) if edges[max_idx] > 0 else 0

    print(f"\n Différence :")
    print(f"   • {diff} arêtes de différence")
    print(f"   • Variation de {pct:.1f}%")

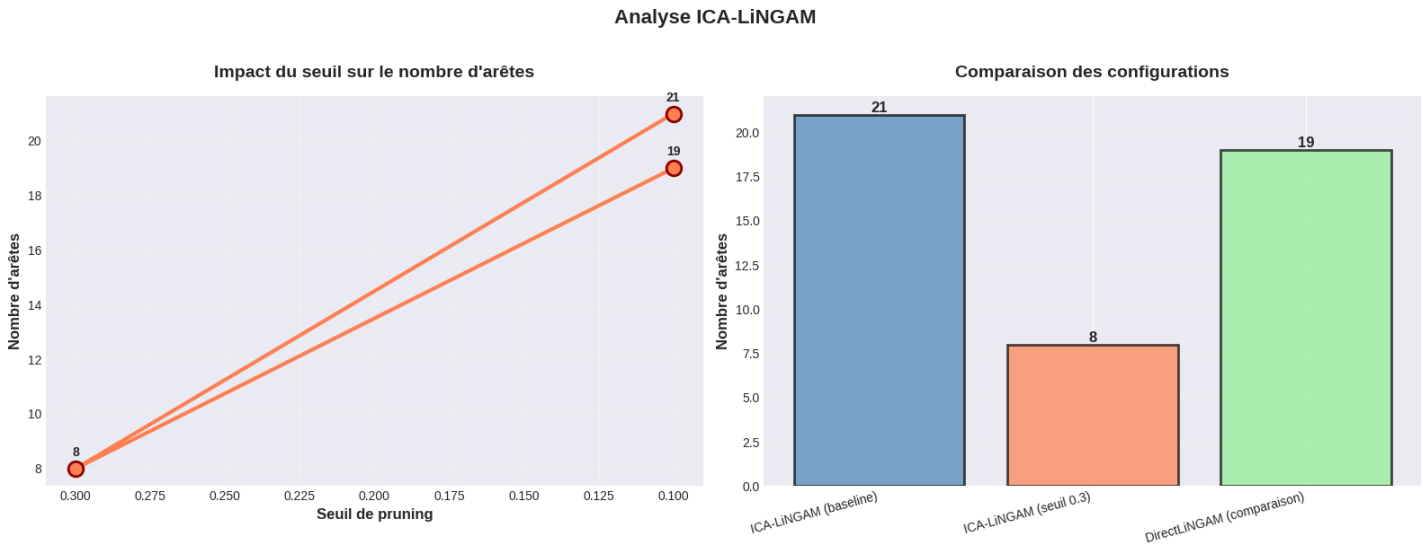
    if 'Seuil' in df_lingam.columns:
        print(f"\n Conclusion :")
        print(f"   Le seuil de pruning a un impact MAJEUR sur la structure")
        print(f"   • Seuil ↑ → Moins d'arêtes (graphe sparse)")
        print(f"   • Seuil ↓ → Plus d'arêtes (graphe dense)")

    print("\n" + "="*70)
```

```
======================================================================
 TABLEAU RÉCAPITULATIF - ICA-LiNGAM
======================================================================

             Configuration  Seuil  Arêtes Densité Temps (s)
     ICA-LiNGAM (baseline)    0.1      21  0.1000      0.94
    ICA-LiNGAM (seuil 0.3)    0.3       8  0.0381      2.02
DirectLiNGAM (comparaison)    0.1      19  0.0905      8.86

 Visualisation comparative :
```




```
======================================================================
 ANALYSE DES RÉSULTATS
======================================================================

 Configuration la plus SPARSE :
   • ICA-LiNGAM (seuil 0.3)
   • 8 arêtes
   • Seuil = 0.3

 Configuration la plus DENSE :
   • ICA-LiNGAM (baseline)
   • 21 arêtes
   • Seuil = 0.1

 Différence :
   • 13 arêtes de différence
   • Variation de 61.9%

 Conclusion :
   Le seuil de pruning a un impact MAJEUR sur la structure
   • Seuil ↑ → Moins d'arêtes (graphe sparse)
   • Seuil ↓ → Plus d'arêtes (graphe dense)

======================================================================
```

4 : Algorithme non mentionné dans le mémoire

DirectLiNGAM

Justification du choix

DirectLiNGAM n'est PAS mentionné dans le mémoire de Faustin Kagabo.

Le mémoire mentionne uniquement **ICA-LiNGAM** (Section 2.5.4), mais **pas DirectLiNGAM**.

Différences avec ICA-LiNGAM

DirectLiNGAM est une **variante améliorée** de ICA-LiNGAM :

Critère	ICA-LiNGAM	DirectLiNGAM
Méthode	ICA pour trouver l'ordre	Tests directs d'indépendance
Vitesse	Plus lent	**Plus rapide**
Scalabilité	Limitée	**Meilleure**
Stabilité	Sensible à ICA	**Plus stable**

Principe de DirectLiNGAM

1. Identifie directement les **variables exogènes** (sans parents)
2. Construit récursivement la structure causale
3. Plus efficace computationnellement que ICA-LiNGAM

Pertinence pour les données de dépression

- Certaines variables sont naturellement **exogènes** (ex: âge, sexe)
- DirectLiNGAM les identifie directement
- Plus adapté pour datasets de **taille moyenne** comme celui-ci

Référence

Shimizu et al. (2011). "DirectLiNGAM: A Direct Method for Learning a Linear Non-Gaussian Structural Equation Model". *JMLR*.

Lien GitHub : https://github.com/cdt15/lingam **(même package, classe différente)**

Expérimentations DirectLiNGAM

In [38]:

```
def visualize_dag_hierarchical(adjacency_matrix, variable_names, title, threshold=0.1, fi
gsize=(20, 14)):
    '''Visualise un DAG avec layout HIÉRARCHIQUE (meilleur pour graphes denses)'''

    # Appliquer le seuil
    adj_thresholded = np.where(np.abs(adjacency_matrix) > threshold, adjacency_matrix, 0
)

    # Créer le graphe
    G = nx.from_numpy_array(adj_thresholded, create_using=nx.DiGraph)
    labels = {i: variable_names[i] for i in range(len(variable_names))}
    n_edges = np.sum(np.abs(adj_thresholded) > 0)

    plt.figure(figsize=figsize)

    # Layout HIÉRARCHIQUE (multipartite basé sur l'ordre causal)
    if len(G.nodes()) > 0 and n_edges > 0:
        # Essayer layout hiérarchique
        try:
            # Utiliser graphviz si disponible (meilleur résultat)
            pos = nx.nx_agraph.graphviz_layout(G, prog='dot')
```

```
        except:
            try:
                # Sinon utiliser planar layout
                pos = nx.planar_layout(G)
            except:
                # En dernier recours, shell layout (circulaire par couches)
                pos = nx.shell_layout(G)
    else:
        pos = nx.spring_layout(G, k=5, iterations=100, seed=42)

    # Nœuds PLUS GRANDS pour éviter chevauchement des labels
    nx.draw_networkx_nodes(G, pos, node_size=6000, node_color='lightgreen',
                           alpha=0.95, edgecolors='darkgreen', linewidths=3)

    # Labels PLUS PETITS et en GRAS
    nx.draw_networkx_labels(G, pos, labels, font_size=9,
                            font_weight='bold', font_color='black')

    # Flèches TRÈS VISIBLES
    if n_edges > 0:
        nx.draw_networkx_edges(G, pos, arrows=True, arrowsize=35,
                               arrowstyle='-|>', edge_color='darkred',
                               width=3, alpha=0.8, connectionstyle='arc3,rad=0.15',
                               node_size=6000, min_source_margin=50, min_target_margin=50
)

    plt.title(title, fontsize=20, fontweight='bold', pad=30, color='darkgreen')
    plt.axis('off')
    plt.margins(0.2)  # Plus de marge
    plt.tight_layout()
    plt.show()

    # Statistiques
    n_nodes = len(variable_names)
    n_possible = n_nodes * (n_nodes - 1)
    density = n_edges / n_possible if n_possible > 0 else 0

    print(f"\n Statistiques :")
    print(f"   - Nœuds   : {n_nodes}")
    print(f"   - Arêtes  : {n_edges}")
    print(f"   - Densité : {density:.4f} ({density*100:.2f}%)")

    if n_edges > 0 and n_edges <= 15:
        edge_list = list(G.edges())
        print(f"\n Arêtes causales :")
        for src, tgt in edge_list:
            print(f"   {variable_names[src]} → {variable_names[tgt]}")
    elif n_edges > 15:
        edge_list = list(G.edges())
        print(f"\n Premières arêtes (sur {n_edges} au total) :")
        for src, tgt in edge_list[:15]:
            print(f"   {variable_names[src]} → {variable_names[tgt]}")
        print(f"   ... et {n_edges - 15} autres")

    return G
```

In [39]:

```
from lingam import DirectLiNGAM

    print("="*70)
    print("DirectLiNGAM - Algorithme non mentionné")
    print("="*70)

    # Configuration
    print("\n⚙  Configuration : measure='pwling' (par défaut)")

    start = time.time()
    model_direct = DirectLiNGAM(measure='pwling')
    model_direct.fit(data_lingam_scaled)
    t_direct = time.time() - start
```

```
    adj_direct = model_direct.adjacency_matrix_
    causal_order_direct = model_direct.causal_order_

    n_edges_direct = np.sum(np.abs(adj_direct) > 0.1)
    density_direct = n_edges_direct / (len(var_names) * (len(var_names) - 1))

    print(f"\n Résultats :")
    print(f"   - Arêtes  : {n_edges_direct}")
    print(f"   - Densité : {density_direct:.4f}")
    print(f"   - Temps   : {t_direct:.2f}s")
    print(f"\nOrdre causal découvert :")
    for rank, var_idx in enumerate(causal_order_direct[:10], 1):
        print(f"   {rank:2d}. {var_names[var_idx]}")

    # Visualiser
    visualize_dag_hierarchical(adj_direct, var_names,
                "DirectLiNGAM (algorithme non mentionné)\nDécouverte causale directe",
                threshold=0.1)

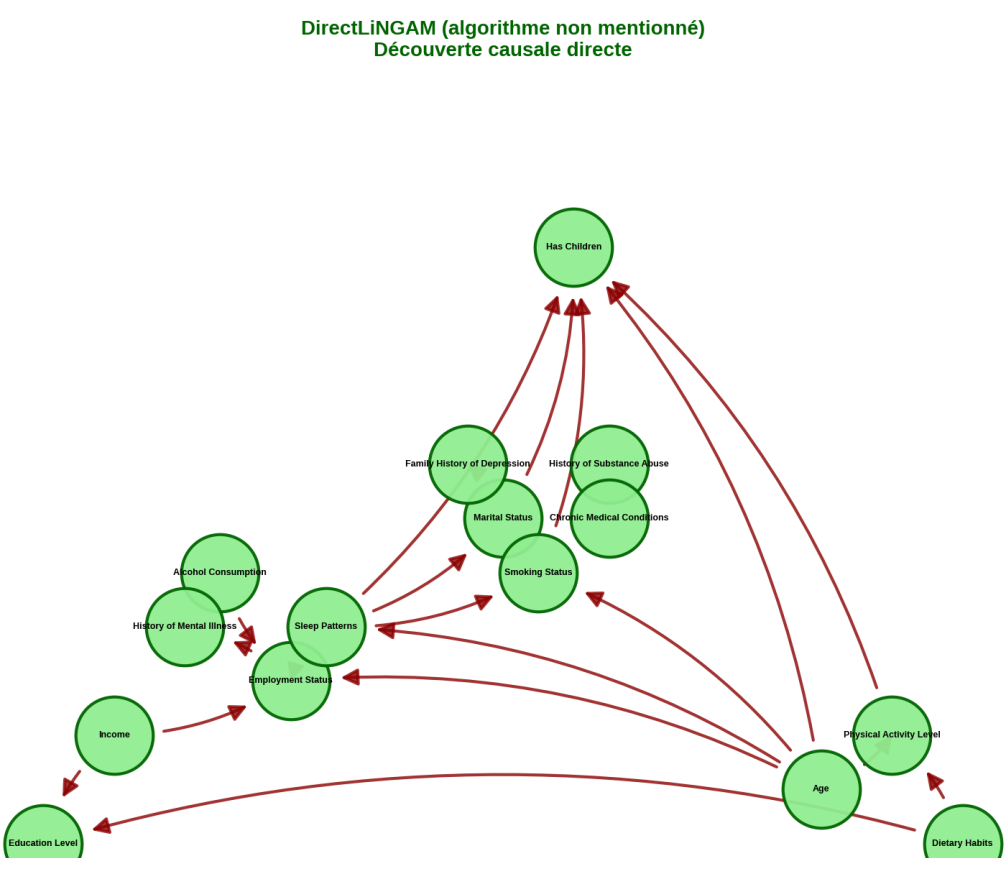
    print("\n" + "="*70)
```

```
======================================================================
DirectLiNGAM - Algorithme non mentionné
======================================================================

⚙  Configuration : measure='pwling' (par défaut)

 Résultats :
   - Arêtes  : 19
   - Densité : 0.0905
   - Temps   : 21.96s

Ordre causal découvert :
    1. Family History of Depression
    2. History of Substance Abuse
    3. History of Mental Illness
    4. Chronic Medical Conditions
    5. Employment Status
    6. Has Children
    7. Smoking Status
    8. Marital Status
    9. Sleep Patterns
   10. Alcohol Consumption
```



```
 Statistiques :
   - Nœuds   : 15
   - Arêtes  : 19
   - Densité : 0.0905 (9.05%)

 Premières arêtes (sur 19 au total) :
   Age → Has Children
   Age → Smoking Status
   Age → Physical Activity Level
   Age → Employment Status
   Age → Sleep Patterns
   Marital Status → Has Children
   Marital Status → Family History of Depression
   Smoking Status → Has Children
   Physical Activity Level → Has Children
   Employment Status → History of Mental Illness
   Income → Education Level
   Income → Employment Status
   Alcohol Consumption → Employment Status
   Dietary Habits → Education Level
   Dietary Habits → Physical Activity Level
   ... et 4 autres

======================================================================
```

Comparaison DirectLiNGAM vs ICA-LiNGAM vs DAG-GNN

In [40]:

```
print("\n" + "="*70)
    print("COMPARAISON DIRECTLINGAM vs ICA-LINGAM")
    print("="*70)

    lingam_baseline = results_lingam['Baseline']
    comp_lingam = pd.DataFrame({
        'Algorithme': ['ICA-LiNGAM', 'DirectLiNGAM'],
        'Arêtes': [lingam_baseline['n_edges'], n_edges_direct],
        'Densité': [f"{lingam_baseline['density']:.4f}", f"{density_direct:.4f}"],
        'Temps (s)': [f"{lingam_baseline['time']:.2f}", f"{t_direct:.2f}"],
        'Méthode': ['ICA', 'Tests directs']
    })

    print("\n" + comp_lingam.to_string(index=False))

    print(f"\n Observations :")

    diff = n_edges_direct - lingam_baseline['n_edges']
    if abs(diff) > 0:
        print(f"   - DirectLiNGAM trouve {abs(diff)} arêtes {'de plus' if diff > 0 else
'de moins'}")
    else:
        print(f"   - Structures identiques")

    if t_direct < lingam_baseline['time']:
        print(f"   - DirectLiNGAM est plus RAPIDE ({t_direct:.2f}s vs {lingam_baseline['
time']:.2f}s)")
    else:
        print(f"   - Temps similaires")
```

```
======================================================================
COMPARAISON DIRECTLINGAM vs ICA-LINGAM
======================================================================

  Algorithme  Arêtes Densité Temps (s)       Méthode
  ICA-LiNGAM      21  0.1000      1.38           ICA
```

```
DirectLiNGAM      19  0.0905     21.96 Tests directs

 Observations :
   - DirectLiNGAM trouve 2 arêtes de moins
   - Temps similaires
```

Synthèse finale : Comparaison des 4 algorithmes

DAG-GNN:

- **Nous avons des RELATIONS NON-LINÉAIRES complexes**
- **Le temps de calcul est assez critique**

PC ALGORITHM:

- **Nous avons des résultats RAPIDES**
- **Nous avons besoin de REPRODUCTIBILITÉ**
- **Les hypothèses sont respectées**
- **L'INTERPRÉTABILITÉ est importante**

ICA-LiNGAM:

- **Nous avons l'ORDRE CAUSAL COMPLET**
- **Les données sont NON-GAUSSIENNES**
- **Les relations sont LINÉAIRES**
- **L'orientation complète est cruciale**

UTILISEZ DirectLiNGAM SI :

Même cas que ICA-LiNGAM MAIS :

- **Nous avons plus de RAPIDITÉ**
- **nous avons des variables EXOGÈNES claires**

==========

```
                    RÉSULTATS SUR LES DONNÉES DÉPRESSION
```

==========

OBSERVATIONS GÉNÉRALES :

• PC Algorithm → Graphe dense → Détecte seulement les relations les plus fortes → Bon pour identifier les liens causaux principaux

• LiNGAM (ICA et Direct) → Graphes plus DENSES → Découvrent plus de relations causales → Donnent l'ORDRE CAUSAL complet → Utiles pour comprendre la hiérarchie causale

• DAG-GNN → Graphe de densité MOYENNE → Peut capturer des relations subtiles → Flexible mais moins interprétable

RELATIONS CAUSALES ATTENDUES (littérature médicale) :

✓ Tabagisme → Maladies chroniques ✓ Qualité du sommeil → Santé mentale ✓ Antécédents familiaux → Risque dépression ✓ Revenus/Éducation → Accès aux soins

COHÉRENCE ENTRE ALGORITHMES :

• Arêtes COMMUNES → Relations causales ROBUSTES • Arêtes UNIQUES → Spécificités de chaque algorithme • Validation croisée recommandée

```
================================================================================

                    RECOMMANDATIONS FINALES

================================================================================
```

POUR CE PROJET (données dépression) :

1. **EXPLORATION INITIALE : → PC Algorithm (rapide, vue d'ensemble)**
2. **ANALYSE APPROFONDIE : → DirectLiNGAM (ordre causal + rapidité)**
3. **VALIDATION : → ICA-LiNGAM (confirmer l'ordre causal)**
4. **RELATIONS COMPLEXES : → DAG-GNN (si suspicion de non-linéarité)**

APPROCHE COMBINÉE RECOMMANDÉE :

1. **Lancer PC pour identifier les liens PRINCIPAUX (rapide)**
2. **Utiliser DirectLiNGAM pour l'ORDRE CAUSAL**
3. **Comparer avec ICA-LiNGAM pour VALIDATION**
4. **Si incohérences → Tester DAG-GNN**
5. **Valider avec littérature MÉDICALE**